

Surveying the Benchmarking Landscape of Large Language Models in Code Intelligence

MOHAMMAD ABDOLLAHI, York University, Canada

RUIXIN ZHANG, York University, Canada

NIMA SHIRI HARZEVILI, York University, Canada

JIHO SHIN, York University, Canada

SONG WANG, York University, Canada

HADI HEMMATI, York University, Canada

Since the release of early Large Language Models (LLMs) such as GPT-2 and GPT-3 around 2020, rapid advancements in LLM capabilities have significantly impacted the field of code intelligence, enabling automation across a wide range of coding tasks, including code generation, program repair, debugging, and software testing etc. As the LLMs are widely used in coding tasks, benchmarking their capabilities in a meticulous and meaningful way is essential.

In this work, we survey 142 related papers published between January 2020 and June 2025, covering 156 unique benchmarks and 32 different coding tasks, to provide a comprehensive review of coding benchmarks to evaluate LLMs, exploring their characteristics, strengths, limitations, dataset construction, task coverage, alignment with real world challenges, and performance metrics.

Our analysis indicates that Python was the most prevalent programming language, featured in 77% of the datasets, and GitHub was the most common data source, used in 46% of the cases. Also, there is an increasing trend in the publication of new benchmark datasets for coding intelligence over the past three years. Prominent publication venues are machine learning conferences such as NeurIPS, ICLR, and ICML, while comparatively fewer datasets have appeared in software engineering venues such as ICSE, ASE, and MSR. In addition, most of the benchmarks focused on the code generation task, with 86 datasets dedicated to this area. We identify key gaps and limitations in existing benchmarks, including bias in datasets, lack of standardized evaluation protocols, and insufficient coverage of real-world challenges. Finally, we present a vision for the future of LLM benchmark creation, advocating for more robust, adaptable, and better alignment with practical code intelligence demands that ensure the reliability and practicality of LLMs in coding workflows.

CCS Concepts: • **Software and its engineering** → **Software maintenance tools**; • **Information systems** → **Information integration**; • **Computing methodologies** → **Natural language processing**; **Artificial intelligence**.

Additional Key Words and Phrases: Benchmark, Code Intelligence, Large Language Models, Software Engineering

1 Introduction

The rapid advancements in Large Language Models have revolutionized the field of code intelligence [61, 155, 168, 211], enabling automation in tasks such as code generation [46, 103, 125], software debugging [88], program

Authors' Contact Information: Mohammad Abdollahi, York University, North York, Canada, moham98@yorku.ca; Ruixin Zhang, York University, North York, Canada, jason666@my.yorku.ca; Nima Shiri Harzevili, York University, North York, Canada, nshiri@yorku.ca; Jiho Shin, York University, North York, Canada, jihoshin@yorku.ca; Song Wang, York University, North York, Canada, wangsong@yorku.ca; Hadi Hemmati, York University, North York, Canada, hemmati@yorku.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/3-ART

<https://doi.org/10.1145/3800957>

repair [17, 85, 201], code translation [137, 153], and software testing [32, 49, 151] etc. These models, trained on vast amounts of code repositories, documentation, and programming discussions, exhibit remarkable capabilities in understanding, generating, and reasoning about code. LLM-powered tools such as GitHub Copilot¹ and Cursor² are already being integrated into modern coding workflows, offering developers intelligent assistance for writing and maintaining code.

Despite the growing adoption of LLMs in code intelligence applications, evaluating their effectiveness, robustness, and applicability in the real world remains a critical challenge [47]. Unlike traditional programming tools, LLM-based approaches operate as black-box models, which take natural language-based task descriptions or code snippets as input and generate probabilistic solutions to solve the tasks; the solutions may contain hallucinations, low effectiveness, security vulnerabilities, or logic errors [66]. Thus, ensuring their reliability requires rigorous benchmarking.

Benchmark datasets play an important role in systematic evaluation and comparison of LLMs in a variety of tasks, programming languages, and complexity levels [19]. However, existing benchmarks vary significantly in their task coverage, data quality, dataset construction methodologies, evaluation criteria, and alignment with real-world scenarios. The lack of standardization and diversity among benchmarks makes it difficult to draw generalizable conclusions about model performance and raises questions about the reliability and representativeness of current evaluation practices. Moreover, the rapid evolution of LLM capabilities has outpaced the development of benchmarks, resulting in many datasets that are no longer sufficiently challenging or comprehensive. There is an urgent need to better understand the landscape of existing benchmarks, assess their strengths and limitations, and identify opportunities for improvement. Therefore, a systematic survey and analysis of benchmark datasets is essential to inform future research, guide dataset creation, and support a more robust and meaningful evaluation of LLMs in code intelligence.

In this paper, we strive to systematically review and characterize the benchmark datasets used to evaluate LLMs in the domain of code intelligence. We focus on capturing the evolution of the benchmarking ecosystem from January 2020 to June 2025. Our analysis is based on 156 unique datasets from 142 studies published at flagship conferences, covering 32 distinct code-related tasks. To guide our investigation, we examine the following research questions (RQs):

- **RQ1: Current Landscape of Benchmark Datasets:** In this RQ, our goal is to systematically characterize existing benchmark datasets in terms of their publication timeline, supported task types, target programming languages, dataset sizes, and task complexity. Additionally, we analyze the structure of the datasets and the venues in which these benchmarks were introduced. This provides a comprehensive view of how benchmarks have evolved and where gaps remain.
- **RQ2: Characteristics and Quality of Benchmark Datasets:** This RQ investigates how benchmark datasets for coding tasks are structured and organized, including their data formats, labeling schemes, and metadata. We examine the quality control mechanisms used during data collection, such as manual validation, filtering, or automated checks. Finally, we assess how well these benchmarks reflect real-world scenarios in terms of relevance, realism, and task representativeness.
- **RQ3: Evaluation Metrics and Techniques:** This question explores the evaluation criteria commonly adopted to measure the performance of LLMs on coding benchmarks. We analyze the types of metrics used, such as accuracy, BLEU, code execution success rate, or functional correctness, and whether they align with the specific goals of the tasks (e.g., code generation, bug repair, or test generation). Furthermore, we investigate whether standardized evaluation protocols or methodologies exist across datasets and how consistently they are applied, highlighting any gaps or inconsistencies in current benchmarking practices.

¹<https://github.com/features/copilot>

²<https://www.cursor.com/>

- **RQ4: Challenges and Limitations:** This RQ focuses on identifying the limitations of existing LLM benchmarking datasets. To investigate this, we began by selecting 14 representative benchmark datasets that collectively present the diversity and scope of the current LLM evaluation landscape in coding intelligence. Next, we analyzed the body of literature that cites and utilizes these benchmarks, extracting and synthesizing the limitations and challenges reported by researchers in their empirical studies. This approach allows us to uncover common concerns related to coverage, bias, applicability, and evaluation reliability, as experienced by the broader research community.
- **RQ5: Future Directions and Improvements:** Building on the findings from RQ4, we outline a set of recommendations and future research directions for the next generation of LLM benchmarks. This includes identifying opportunities for crafting datasets more comprehensive and realistic. We also discuss the potential for incorporating new data types, interactive tasks, and multi-modal contexts to better reflect real-world development environments.

This article makes the following contributions:

- A systematic review of 156 datasets from 142 papers that are created to evaluate LLMs for coding intelligence, covering 32 distinct tasks.
- A detailed taxonomy and comparative analysis of characteristics, structures, evaluation metrics, and task coverage.
- An assessment of quality control practices and real-world relevance across existing benchmarks.
- An identification of key limitations in current benchmarking practices, drawn from the literature analysis of representative datasets.
- A set of actionable recommendations for the next generation of benchmarks to better reflect the evolving capabilities of LLMs and the practical needs of developers.
- We have released the list of the papers surveyed and our scripts to facilitate replication of this study³.

We believe that this work is valuable for both researchers and practitioners, as it provides a structured understanding of the current benchmarking landscape, highlights gaps and inconsistencies in existing evaluations, and offers practical guidance for designing, selecting, and applying benchmark datasets to assess LLM capabilities. This work will serve as a foundation for the advancement of more meaningful, reproducible, and representative LLM evaluation in the code intelligence research community.

The rest of the article is organized as follows. Section 2 provides background information and reviews related work. Section 3 outlines the research methodology proposed in this article. Section 4 addresses the RQs and presents the corresponding results. Section 5 discusses potential threats to the validity of this study. Finally, Section 6 presents the conclusion and suggests future directions.

2 Background and Related Work

In this section, we first provide a background on the application of LLMs across various code intelligence tasks. Subsequently, we review related surveys, highlighting the distinctions between their scope and the focus of our study.

2.1 Background on LLMs for Coding Intelligence

Since the release of early large language models such as GPT-2 [144] and GPT-3 [18] around 2020, LLMs have been increasingly integrated into the code intelligence domain, transforming the way developers approach complex challenges. These models are trained on extensive datasets that include both natural language and source code, giving them the ability to interpret, generate, and adapt text and code. Due to this versatility, LLMs are now

³https://github.com/mohammad-abdollahi/survey_llm_benchmark_dataset

used in many tasks in code intelligence [61, 136, 211]. A major area where LLMs are making an impact is code generation. Developers can provide natural language prompts, and LLMs can produce the corresponding code snippets, reducing repetitive tasks and speeding up development. Beyond simply writing code, these models can also help with code summarization by turning large or complex functions into clear, concise explanations that are easier for teams to understand and maintain [47, 67]. LLMs are also valuable for code translation. As projects evolve, it is necessary to convert the code from one language or framework to another. LLMs can help by generating initial translations that maintain the intended functionality, reducing manual errors and saving time during migrations [219]. Software testing is another domain that benefits from LLM integration [168]. These models can assist in creating unit tests, generating assertions, and even automating the creation of test cases from requirements and existing code. This leads to stronger quality assurance practices, helping teams detect and resolve issues earlier in the development cycle [135]. They are also used for vulnerability detection, scanning code to identify security risks, and helping developers build safer software [223].

2.2 Related Work

Several survey papers have explored LLMs and benchmark datasets in the context of software engineering, which is related to our survey. In this section, we analyze the existing literature based on a set of key aspects, summarized in Table 1. These aspects include the timeframe scope of datasets, data source, data quality considerations, LLM-specific focus, current landscape, key characteristics analysis, benchmark limitations and shortcomings, evaluation pipelines and metrics, future directions, and data contamination.

Our survey is distinct in that it only includes post-GPT-2 era (i.e., 2020 to 2025) benchmarks, whereas other recent surveys often review a significant number of datasets from the pre-LLM era. We acknowledge that legacy benchmarks (e.g., Defects4J [80], CodeSearchNet [70], QuixBugs [105], and CoNaLa [199]) are valuable and often serve as baselines; However, we exclude these datasets to focus specifically on benchmarks developed in the Generative AI era, as our objective is to analyze the current landscape of LLM benchmarking. Furthermore, these datasets have been covered extensively in previous software engineering surveys [64, 170]. Data source refers to whether the survey discusses the origin of the datasets. Data quality considerations evaluate whether the survey addresses criteria such as filtering and validation in the construction of benchmarks. The LLM-specific focus indicates whether the survey is focused on LLMs or otherwise broadly includes other AI or machine learning models. The current landscape evaluates whether the survey provides a broad overview of existing benchmark datasets by reporting general features such as the programming languages covered, the types of tasks, and the size of datasets. Key benchmark characteristics analysis assesses whether the survey provides insights into key dataset properties such as task diversity, coding patterns, and structure. The aspect of benchmark limitations and shortcomings considers whether the survey discusses issues such as data leakage, representativeness, and coverage gaps. The evaluation pipelines and metrics refer to whether the survey includes a review of evaluation methodologies, test setups, and performance metrics specific to LLMs and their tasks. Future direction evaluates whether the survey offers recommendations for future benchmark development. This includes suggestions for improving the quality of datasets and providing approaches for constructing robust, diverse, and meaningful benchmarks that better reflect the complexity of real-world software engineering tasks. Finally, data contamination examines whether the survey systematically addresses the critical issue of data leakage and evaluates whether the survey reviews mitigation strategies and detection methods to ensure the validity of performance results.

Although most existing surveys address some of these aspects to a limited extent, none offer comprehensive coverage across all dimensions. In contrast, our survey systematically and thoroughly examines each aspect through in-depth analysis of the underlying data, offering a more holistic perspective.

Among the existing literature, the most closely related studies to ours are the surveys by Hu et al. [64] and Wang et al. [170]

Table 1. Comparison of contributions between our survey and the existing related surveys/reviews.

No	Studies	Time Frame of Survey	Data Source	Data Quality	LLM Focus	Current Landscape	Key Characteristics Analysis	Limitations	Evaluation	Future Directions	Data Contamination
1	Hu et al. [64]	2014-2025	✓	×	✓	✓	×	✓	✓	✓	×
2	Wang et al. [170]	2018-2024	✓	×	✓	✓	×	✓	×	✓	×
3	Koohestani et al. [83]	2014-2025	×	×	×	✓	×	✓	×	×	×
4	Zhu et al. [223]	2005-2025	✓	✓	×	✓	✓	✓	×	✓	×
5	Cao et al. [19]	2014-2024	✓	✓	✓	×	×	✓	×	✓	×
6	Chen et al. [30]	2021-2025	✓	×	✓	×	×	✓	×	×	✓
7	Zhou et al. [221]	2017-2025	✓	×	✓	×	×	✓	×	×	✓
8	Zheng et al. [218]	2020-2024	×	×	✓	✓	×	×	✓	×	×
9	Chen et al. [28]	2021-2024	×	×	✓	×	×	×	✓	×	×
10	Zhang et al. [214]	2021-2024	×	×	✓	×	×	×	×	×	×
11	Lin et al. [78]	2021-2024	×	×	×	×	×	×	×	×	×
12	Our survey	2020-2025	✓	✓	✓	✓	✓	✓	✓	✓	✓

Although Hu et al. [64] provide a broad systematic review of 291 benchmarks for evaluating LLMs in software engineering, focusing on their construction methods, evaluation metrics, and future challenges, our article goes further by providing a more granular analysis of benchmark construction processes, rigorously assessing data sources and quality control methods, real-world representativeness, data contamination mitigation measures, systematically categorizing diverse evaluation pipelines, and propose a robust evaluation suite. Unlike their high-level survey, we explicitly identify the shortcomings drawn from studies that actually used these benchmarks, offering a more precise, evidence-backed, and generalizable framework to address critical gaps in LLM benchmarking datasets for software engineering alongside providing a unified framework for creating benchmarks. Wang et al. [170] conduct a survey of existing benchmark datasets for CodeLLMs, categorizing them by software engineering task throughout the software development life cycle. In contrast, our work goes beyond listing benchmarks to analyze their creation, quality controls, shortcomings and evaluation aspects in depth, offering a more comprehensive framework for assessing dataset quality and real-world applicability.

Koohestani et al. [83] present a comprehensive review of 204 AI-for-Software-Engineering (AI4SE) benchmarks drawn from 173 studies, focusing on the operational challenge of discoverability. Their primary contribution is BenchScout, a search tool, and BenchFrame. While their work is instrumental for locating datasets across the broad AI4SE spectrum, it does not aim to provide a critical landscape analysis of LLM-specific evaluation. Our work complements this by shifting the focus from searchability to critical analysis. In addition, we provide a deeper analysis of benchmark creating and quality control, benchmark shortcomings and evaluation practices, including construction processes, quality control, data contamination, and dataset characteristics specific to LLM performance assessment.

Zhu et al. [223] concentrate solely on benchmarks for software defect detection. Their survey offers a detailed classification of 132 datasets based on defect types, domains, and construction methods. In contrast, our study covers benchmarks across a wide range of software engineering tasks, not just defects, offering a broader perspective that includes classification, generation, recommendation, and more.

Several other studies focus on the limitations in LLM benchmarking. Cao et al. [19] introduce HOW2BENCH, a 55-criterion checklist for improving code-related benchmarks, identifying widespread issues in data quality, reproducibility, and transparency. While their primary contribution is a guideline for benchmark creation, our work concentrates on analyzing the sources, characteristics, and data quality metrics of existing benchmarks, supported by a comprehensive dataset-level review.

Similarly, Chen et al. [30] and Zhou et al. [221] address the problem of data contamination and leakage. Chen et al. [30] discuss the transition from static to dynamic benchmarking approaches and propose criteria to mitigate contamination. Zhou et al. [221] introduce LessLeak-Bench, employing near-duplicate detection and manual inspection to quantify and reduce leakage in 83 benchmarks. While these works provide valuable insights into a specific subset of limitations, our survey takes a more holistic approach by cataloging a wide range of benchmark shortcomings—including, but not limited to, data leakage—and systematically documenting them across multiple

benchmark datasets. Also, we systematically analyze and categorize the diverse data contamination mitigation measures adopted across the benchmarks.

Zheng et al. [218] present a comprehensive survey of Code LLMs, analyzing their evolution, categorizing them by developers, and comparing their performance with general LLMs across various software engineering tasks. While their work provides valuable insights into the capabilities and development of Code LLMs, their treatment of benchmarks is limited to a high-level overview. In contrast, our paper focuses specifically on benchmark datasets, offering a more in-depth and systematic analysis of their construction, characteristics, evaluation protocols, and limitations. Chen et al. [28] offer a comprehensive overview of LLM evaluation methods for code generation, including metrics such as correctness, efficiency, readability, and user satisfaction. While their study is centered on evaluation perspectives, our work extends this by providing a complete taxonomy of evaluation metrics, categorized by type and usage, and mapping them to specific benchmark tasks and phases.

Jin et al. [78] present a survey of LLMs and LLM-based agents in software engineering, covering six key areas from requirement engineering to software maintenance. Their emphasis on agent-based reasoning and task distribution is valuable, but their benchmark analysis is primarily from the perspective of autonomous behavior. Our study, by contrast, is focused explicitly on the datasets and evaluation frameworks for LLMs themselves, incorporating agentic behavior as one of many dimensions rather than the focal point.

Finally, Zhang et al. [214] provide a broad overview of language models for code, tracking the shift from statistical models to transformer-based architectures and discussing structural features. Although they mention benchmark datasets and basic attributes such as size and programming language, their emphasis remains on model development and performance. In contrast, our work offers a detailed examination of benchmark features, including construction methodology, dataset characteristics, evaluation protocols, and quality considerations, specifically aimed at understanding and improving LLM evaluation in software engineering contexts.

In summary, while existing studies offer valuable insights into specific aspects of LLM and software engineering benchmarks, none provides a comprehensive and fine-grained analysis that systematically covers benchmark construction, characteristics, data quality, evaluation methodologies, and limitations across a diverse range of code intelligence tasks in software engineering. Our survey fills this gap by offering a unified and detailed view.

3 Methodology of Systematic Survey

3.1 Sources of Information

In this paper, we follow the guidelines of Keele et al. [44] and Petersen et al. [81] to conduct the survey, to collect and analyze research on LLM benchmark datasets for code intelligence tasks in software engineering published between January 2020 and June 2025, i.e., a period that notably begins with the release of LLM models such as GPT-2⁴. We think this time frame is particularly appropriate for reviewing the evolution of data-driven evaluation frameworks for several reasons: a) increased adoption of LLMs for code intelligence tasks in software engineering. In recent years, the integration of LLMs into various code intelligence tasks has prompted the development of more specialized benchmark datasets. This surge reflects the industry's growing reliance on data-driven evaluation to ensure model robustness and effectiveness. b) advancements in LLM capabilities and evaluation methodologies. The period from 2020 to 2025 has witnessed rapid progress in both the performance of LLMs and the sophistication of evaluation metrics, facilitating more accurate and comprehensive assessments of model performance in code intelligence. To gather the corpus of relevant literature, we systematically searched for research papers across four major databases, including ScienceDirect, IEEE Xplore, ACM Digital Library, and Google Scholar. Figure 1 presents the workflow of the methodology adopted for study identification and selection.

⁴<https://openai.com/index/gpt-2-1-5b-release/>

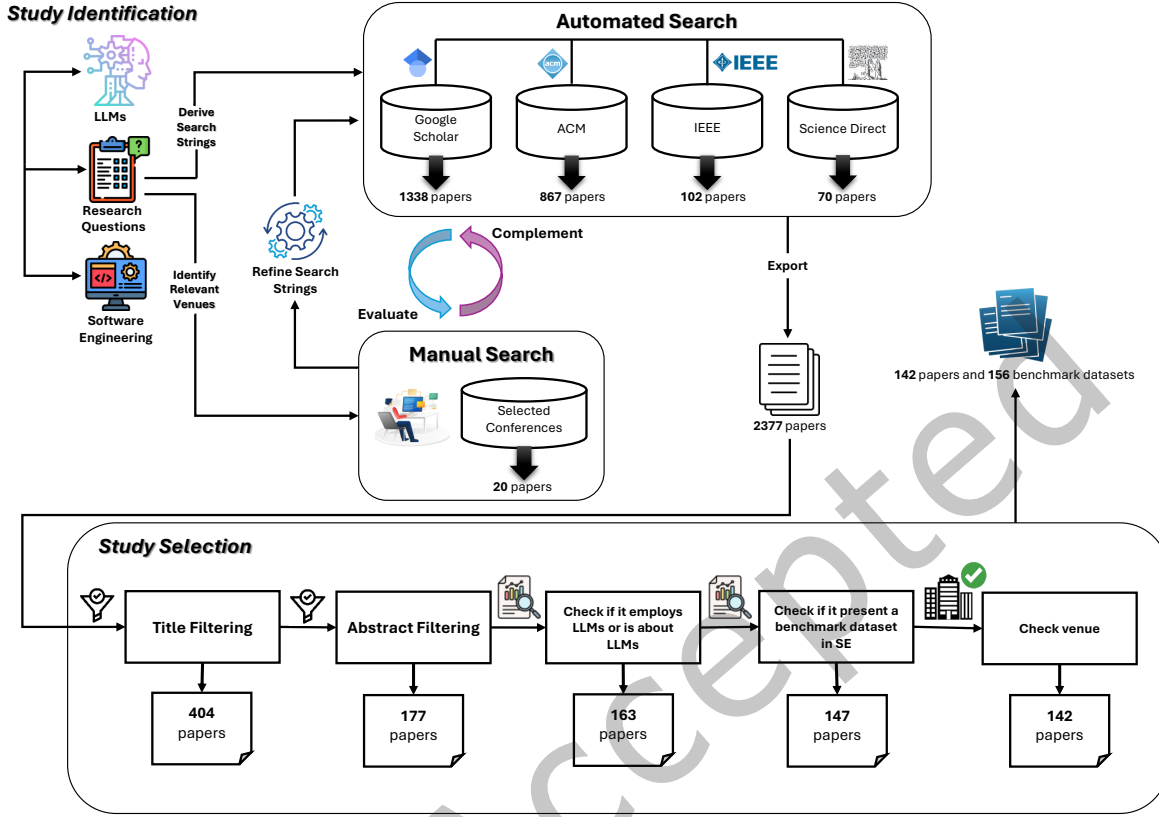


Fig. 1. The workflow of our survey

3.2 Search Terms

From earlier work [6, 67, 78], we identify key phrases used in the search and our experience with the subject area. The following are the search terms:

search terms

(Benchmark Dataset OR Benchmarking) AND (LLM OR Large Language Model) AND (Code Intelligence OR Coding Task OR Code Generation OR Code Summarization OR Test Cases Generation OR Patch Generation OR Code Optimization OR Code Translation OR Program Repair OR Requirement Generation OR Software Development OR Software Engineering OR Code Review OR Code Analysis OR Code Completion OR Code Understanding)

We also employed a snowballing approach to expand our set of search terms, aiming to improve the coverage and relevance of our literature search. Specifically, after retrieving an initial set of studies using predefined keywords, we systematically reviewed their references (backward snowballing) and the subsequent papers that cited them (forward snowballing). Through this process, we identified additional relevant keywords and terminology used in the context of LLM benchmarking in code intelligence. These newly discovered terms were

then incorporated into our search strategy to ensure that we captured a more comprehensive and representative set of studies for our analysis.

3.3 Study Selection and Quality Assessment

The process of selecting studies to be included in our survey on LLM benchmark datasets involved the following stages: (1) an initial screening based on the study titles, (2) a secondary selection after reviewing the abstracts, and (3) a final decision after a thorough reading of the full articles. It is important to note that the initial search results included many entries that were not relevant to benchmark datasets for large language models in code intelligence. This discrepancy was primarily due to incidental keyword matches. To ensure the quality and relevance of our dataset, each study was manually reviewed, and unrelated entries were removed. Additionally, we observed several duplicate entries, as the same study was sometimes indexed by multiple sources. These duplicates were manually identified and excluded.

The initial inclusion criteria for our survey are as follows: (1) studies must be either peer-reviewed publications or recent arXiv preprints published between late 2023 and 2025 that have not yet completed the peer-review process; However, preprints must explicitly pass the rigorous quality assessment outlined in Section 3.3.1, specifically satisfying criteria regarding validation methodology, data integrity, and evaluation soundness. We include these qualified preprints to ensure coverage of the latest developments, (2) studies must report experimental results, (3) studies must introduce a novel benchmark dataset or aim to improve or extend existing benchmarks, and (4) benchmark datasets must either utilize LLMs directly or are employed as baselines for comparison in studies about benchmark datasets involving LLMs. We also applied the following exclusion criteria to filter out irrelevant studies: (1) studies focused on unrelated engineering fields (e.g., electrical engineering, hardware engineering, quantum physics), (2) studies published only in workshops, (3) survey or review articles, (4) studies centered on traditional natural language processing techniques without the use of large language models, and (5) publications categorized as books, book chapters, tutorials, or technical reports.

3.3.1 Title Filtering Stage. In the initial filtering stage, we examined the titles of the studies to determine their potential relevance to LLM benchmark datasets in code intelligence. Given that titles often provide limited context about the full scope of the study, our evaluation in this phase was guided primarily by the presence of predefined keywords. The guiding research question for this stage was: Do the titles contain specific keywords or phrases indicative of benchmark datasets for large language models in the domain of code intelligence? For example, while a title such as “Benchmarking Large Language Models for Automated Verilog RTL Code Generation” contains several relevant terms, namely, “benchmarking”, “large language model”, and “code generation”, a closer inspection reveals that the study is related to hardware design rather than code intelligence or software engineering. Such studies were excluded. Through manual selection of 2,377 records, we identified 404 unique studies that met our preliminary inclusion criteria and were retained for further review.

Abstract Filtering Stage: Building on the results of the title screening, we proceeded to a more detailed examination of the abstracts. For each study, the abstract was segmented into four key components: context, objective, approach, and results/findings, in alignment with the common scientific abstract structure. This decomposition enabled a more precise evaluation of each study’s relevance to our research objectives. As a result of this stage, we retained 177 unique studies for subsequent verification and inclusion in our systematic review.

Content Filtering Stage: In this stage, we conducted an in-depth analysis of the full text of each candidate study to ensure alignment with the objectives of our review. To systematically assess the relevance and quality of the identified papers, we developed a set of evaluative criteria in the form of structured questions. These questions were designed to verify the suitability of each study in the context of benchmarking LLMs for code intelligence tasks. A study was retained for further analysis only if it provided satisfactory answers to 75% of the following questions:

- (1) Is there a clearly stated research objective related to the construction of a benchmark dataset for evaluating LLMs in code intelligence?
- (2) Does the proposed benchmark make use of LLMs either in its construction or evaluation?
- (3) Is the benchmark dataset designed to apply to a variety of LLMs?
- (4) Does the study present an explicit contribution to specific code intelligence tasks?
- (5) Is there a clear methodology for validating the effectiveness of the proposed dataset?
- (6) Is the dataset constructed from a reliable and credible source of information?
- (7) Are there documented steps to ensure the quality and integrity of the dataset?
- (8) Does the dataset contribute to a well-defined task within the domain of code intelligence?
- (9) Is the dataset structure suitable for evaluation or fine-tuning of LLMs?
- (10) Does the dataset maintain relevance, representativeness, and realism with respect to real-world code intelligence scenarios?
- (11) Are the evaluation metrics employed in the study appropriate for assessing task-specific performance and aligned with the research objectives?
- (12) Are the evaluation pipelines and methodologies sound, valid, and appropriate for assessing LLM performance?
- (13) Do the results presented in the study align with the stated research objectives, and are they reported clearly and coherently?

Studies that failed to meet these criteria were excluded from further consideration. Through this rigorous filtering process, we identified a final set of 142 primary studies that directly contribute to our research questions. Duplicate entries and studies lacking methodological rigor or relevance to LLM benchmark datasets in software engineering were removed at this stage. This refined corpus forms the basis for the subsequent analysis presented in this paper.

3.4 Inter-Rater Reliability

To mitigate subjective bias and ensure the reliability and reproducibility of the study selection process, two researchers independently categorized the articles in the title, abstract, and full-text filtering stages. Following the independent review, we assessed the consistency between the reviewers to validate the integrity of the dataset selection. The reliability between the researchers was evaluated using two metrics: overall agreement and Cohen's Kappa coefficient (κ). The process yielded an overall agreement rate of 0.95. Furthermore, the calculated Cohen's Kappa was 0.89, indicating a high level of agreement according to standard interpretation guidelines. Despite the high level of agreement, disagreements occurred in specific cases. To mitigate subjectivity and ensure quality, a third researcher acted as an arbitrator to resolve conflicts that could not be resolved between the two primary reviewers. This triaged approach helped minimize classification bias and improved the overall consistency of the final survey corpus.

It should be noted that five benchmarks (i.e., ConDefects [183], GitHub-Recent-Bugs [89], BinPool [12], ICVul [118], and VulZoo [150]) did not include LLM-based evaluations in their original publications. We made a strategic decision to include these datasets because they were explicitly designed to resolve critical validity threats in LLM benchmarking, most notably data leakage and domain-specific scarcity. ConDefects and GitHub-Recent-Bugs address the contamination problem by strictly curating samples from repositories created after the training cut-offs of major models, thereby ensuring the availability of unseen data for realistic evaluation. BinPool, ICVul, and VulZoo were included to address critical gaps in binary analysis and vulnerability detection, areas in which high-quality benchmark datasets remain scarce. These benchmarks provide large-scale, meticulously cleaned

data with fine-grained metadata and reproducibility guarantees that are essential for assessing rigorous security-focused LLM agents. Their inclusion ensures our survey covers the frontier of robust and contamination-free evaluation resources.

4 Result Analysis

4.1 RQ1: What Is the Current Landscape of Benchmark Datasets

This RQ targets understanding the current landscape of publications. We examined publication dates, venues, types of coding tasks, programming languages, dataset size, evaluation metrics, and benchmark structure in which they were presented.

4.1.1 Publication Date. To assess the current state of benchmark dataset research in code intelligence with LLMs, we analyzed the publication years of 156 benchmark-related papers. These benchmarks span a variety of code intelligence tasks, including code generation, debugging, test case synthesis, issue resolution, and more. The temporal distribution of these articles reveals a strong upward trend in recent years, indicating a greater focus on research and investment in the development of datasets for evaluating LLMs across diverse code intelligence tasks. Figure 2 presents the chronological order of the datasets reviewed in our work.

Figure 3 shows the annual publication trends for benchmark datasets from January 2020 to June 2025. Activity in the early years was limited as GPT-3 was released in mid-2020 and no LLM-specific benchmarks were observed that year. In 2021, five benchmark datasets were introduced, followed by modest growth in 2022 with 12 publications. A significant increase occurred in 2023, with 24 benchmark papers published, marking the beginning of a sharp upward trend in dataset creation. This growth accelerated in 2024 and the first half of 2025, which saw the release of 75 and 40 benchmark datasets, respectively, the highest counts in the time frame analyzed, accounting for more than half of all publications included in our survey. The figure suggests a continuing upward trajectory in the development of LLM-focused benchmarks for code intelligence tasks.

This pattern reflects the rapid expansion of interest in developing standardized evaluation frameworks for LLMs in code intelligence. The increase in benchmark activity aligns with the widespread adoption of LLMs across multiple software tasks, necessitating robust, task-specific, and general-purpose benchmarks to evaluate model performance effectively. The sharp increase since 2023 probably results from the growing availability of powerful LLMs and the increasing recognition of benchmark datasets as a foundational component for fair and reproducible evaluation in the field. Thus, our analysis indicates that benchmark dataset development has become a central focus in the LLM-for-code-intelligence research community.

4.1.2 Coding Tasks and Complexity. The surveyed benchmark datasets span a diverse array of coding tasks, encompassing both foundational and emerging challenges. Table 2 outlines the datasets, their corresponding tasks and the software development lifecycle (SDLC) phases. These phases span from Requirements & Design and Implementation to Comprehension, Testing, Maintenance, and Security. Within these phases, tasks include, but are not limited to, code generation, code translation, completion, understanding, automated program repair, bug repair, test case generation, fault localization, issue resolution, and software engineering question answering. Among these, code generation dominates the landscape, accounting for approximately 52.8% of all tasks, followed by code translation (7%), code completion, test case generation, and vulnerability detection. Other tasks, such as automated program repair, debugging, and fault localization, appear with moderate frequency, while more specialized tasks, such as formal software verification, mobile agent evaluation, and code optimization, are relatively rare.

A notable trend is the shift in task complexity and granularity. Early benchmarks predominantly focused on limited and single-function code generation problems [14, 29]. In contrast, recent benchmarks embrace broader, real-world tasks such as class-level generation, repository-level reasoning, multi-turn debugging, and



Fig. 2. Chronological overview of LLM benchmark datasets by task

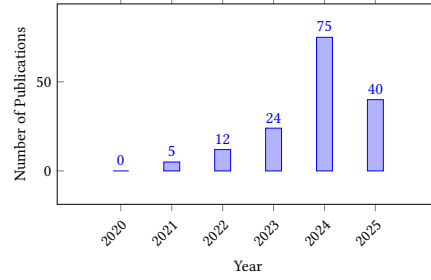


Fig. 3. Publication trend of LLM benchmark datasets (2020–2025).

agent-based workflows. For example, newer benchmarks such as Codev-Bench [138] and CoderEval [202] focus on class-level and project-level generation tasks, requiring models to handle more complex code structures and broader contextual reasoning across components. Similarly, datasets such as StackEval [154] adopt a multi-task setup, evaluating LLMs across a diverse set of tasks such as code generation, debugging, code review, and software engineering question answering. The cooccurrence of these tasks within a single benchmark allows for a broader assessment of a model’s generalization and versatility across distinct coding challenges. This shift toward multi-faceted evaluation reflects the growing need to benchmark LLMs beyond narrow, task-specific capabilities.

Importantly, the collection of datasets reveals that 130 benchmarks focus on a single task, while 26 benchmarks support multiple tasks, often combining generation with debugging, translation, or evaluation subtasks. For example, CodeXGLUE [120] includes 14 datasets for 10 different code intelligence tasks. Table 3 depicts the distribution of datasets for single and multi-task scenarios. This growing inclusion of multi-task, multi-modal, more realistic, and complex benchmarks reflects the increasing demand to evaluate LLMs in isolation and as components of integrated code intelligence workflows that better reflect the challenges faced by practitioners in real-world coding scenarios.

4.1.3 Dataset Context Scope: From Single-Function to Repository-Level Contexts. The table 4 depicts the detailed distribution of context scopes for different datasets. One of the most significant evolutions in coding benchmarks is the expansion of context scope (i.e., the amount of surrounding code or documentation an LLM must process to solve a task). To categorize the varying levels of information available to models, we define four distinct levels of context scope. These levels represent a hierarchy of complexity that ranges from isolated algorithmic units to real-world repository environments.

- **Single-Function Scope:** The most basic level focuses on isolated algorithmic logic. Tasks in this category involve a standalone function based on a docstring or signature, with no external dependencies or global state.
- **Class-Level Scope:** Introduces Object-Oriented Programming complexities. Tasks require generating code that maintains internal state and interacts correctly with member variables and other methods within a specific class definition.
- **Multi-File Scope:** Focuses on modularity and inter-file communication. Tasks involve handling explicit dependencies, requiring the correct import and utilization of API calls or data structures defined in external files.
- **Repository-Level Scope:** The most realistic and complex level, simulating a full software environment. Tasks require navigating a complete codebase to identify implicit dependencies, necessitating the use of advanced algorithms or autonomous agents to effectively handle the extensive context.

Table 2. Distribution of code intelligence tasks across benchmark datasets, categorized by Software Development Lifecycle phases.

SDLC Phase	Coding Task	#Studies	#Datasets	References
Requirements & Design	API Generation	3	3	[95, 160, 220]
	Log Statement Generation	1	1	[162]
	Software Requirement Elicitation	2	2	[56, 166]
	Software Specification Generation	1	1	[98]
Implementation	Code Generation	75	86	[13, 14, 16, 20, 22–25, 27, 29, 33, 34, 41, 42, 48, 55, 57–60, 63, 65, 68, 72, 73, 82, 84, 87, 91, 93, 94, 97, 99, 100, 102, 104, 110, 111, 113, 115, 120, 122, 129–132, 139–142, 146, 154, 156, 167, 171, 176, 178–181, 185–190, 200, 202–204, 207, 216, 217, 224, 225]
	Code Translation	10	13	[4, 76, 82, 120, 132, 134, 174, 190, 191, 217]
	Code Completion	9	9	[16, 23, 40, 48, 116, 120, 138, 194, 224]
	General Software Development	2	2	[90, 129]
	Mobile Agent Evaluation	1	1	[38]
	Code Editing	3	3	[45, 86, 159]
	Code Optimization	1	1	[190]
	Code Summarization	3	3	[120, 132, 190]
	Code Refinement	1	1	[120]
	Clone Detection	1	1	[120]
	Code Search	1	1	[120]
	Code Retrieval	1	1	[116]
	API Recommendation	1	1	[48]
	Code Understanding	5	5	[23, 53, 121, 128, 215]
Comprehension	SE Question Answering	2	2	[62, 154]
	Formal Software Verification	1	1	[117]
	Code Reasoning	3	3	[26, 31, 53]
	Code Execution	2	2	[53, 72]
Testing	Test Case Generation	9	9	[7, 15, 48, 71, 126, 172, 175, 207, 212]
	Test Case Repair	3	3	[112, 145, 152]
Maintenance	Automated Program Repair	8	8	[11, 75, 82, 114, 158, 163, 183, 192, 207]
	Debugging	4	5	[89, 154, 190, 196]
	Issue Resolving	6	6	[8, 77, 124, 193, 205, 206]
	Bug and Defect Detection	5	5	[114, 120, 133, 196, 207]
	Fault Localization	8	8	[11, 16, 35, 89, 114, 183, 196, 222]
	Code Review	3	3	[107, 154, 190]
	Commit Message Generation	1	1	[16]
	Vulnerability Detection	9	9	[5, 12, 43, 101, 109, 118, 150, 161, 198]
Unique Total	32	142	156	-

Table 3. Task scope of benchmark datasets, i.e., Single-Task vs. Multi-Task.

Coding Task	# Datasets	# Studies	References
Single Task	130	118	[4, 5, 7, 8, 12–15, 20, 22, 24–27, 29, 31, 33–35, 38, 40–43, 45, 55–60, 62, 63, 65, 68, 73, 75–77, 84, 86, 87, 90, 91, 93–95, 97–102, 104, 107, 109–113, 115, 117, 118, 121, 122, 124, 128, 130, 131, 133, 134, 138–142, 145, 146, 152, 156, 158–163, 166, 167, 171, 172, 174–176, 178, 180, 181, 187–189, 191–194, 198, 200, 202–206, 212, 215, 216, 220, 222, 225]
Multi-Task	26	24	[11, 16, 23, 48, 53, 71, 72, 82, 89, 114, 116, 120, 126, 129, 132, 154, 179, 183, 185, 190, 196, 207, 217, 224]
Unique Total	156	142	-

Single-function benchmarks dominate and focus on isolated algorithmic tasks. Repository-level benchmarks come next, with 39 datasets that address complex project structures. The Class level and multi-file scopes remain marginal, with 6 and 11 datasets, respectively.

Table 4. Distribution of benchmark datasets by Code Context Granularity

Granularity	# Datasets	# Studies	References
Function Level	100	86	[4, 5, 13, 13, 13, 14, 14, 22, 22, 23, 27, 29, 31, 33–35, 41, 43, 45, 48, 53, 55, 56, 58–60, 63, 65, 72, 73, 73, 75, 76, 82, 87, 91, 93, 95, 97, 100, 107, 110, 110–112, 114, 117, 118, 120–122, 128, 130–133, 139–141, 146, 150, 152, 154, 154, 154, 156, 158, 160–163, 166, 167, 172, 176, 178, 180, 181, 183, 185, 186, 188, 190, 191, 191, 191, 191, 192, 196, 200, 203, 203, 203, 204, 204, 215, 217, 220, 224, 225]
Class Level	6	6	[15, 26, 42, 84, 212, 216]
Multi File	11	11	[12, 24, 25, 38, 68, 71, 98, 109, 145, 159, 189]
Repo Level	39	39	[7, 8, 11, 16, 20, 40, 57, 62, 77, 86, 89, 90, 94, 99, 101, 102, 104, 113, 115, 116, 124, 126, 129, 134, 138, 142, 171, 174, 175, 179, 187, 193, 194, 198, 202, 205–207, 222]
Unique Total	156	142	-

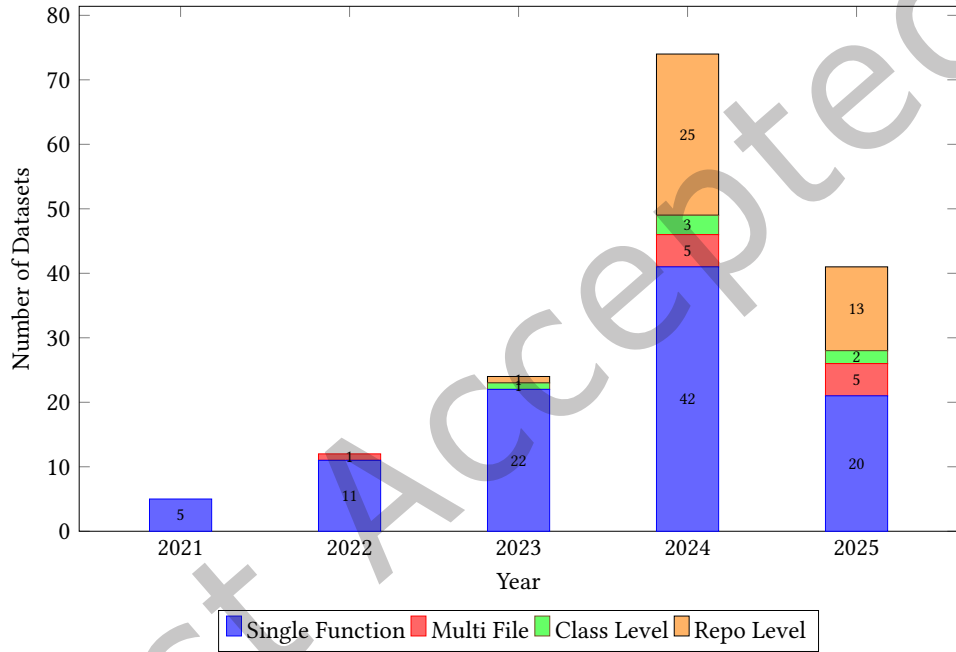


Fig. 4. Evolution of code context scope in LLM benchmark datasets (2021–2025)

Figure 4 highlights the shift from single-function benchmarks to more complex repository-level benchmarks, and we observe a clear paradigm shift toward realistic evaluation. The period from 2021 to 2023 was characterized by single-function tasks, which accounted for almost all benchmarks. 2024 marked an inflection point, with repository-level datasets surging from 1 to 25, driven by advances in LLMs. 2025 data indicate stabilization, with repository-level contexts becoming a standard requirement alongside traditional single-function evaluations.

4.1.4 Interaction Modality: Single-Turn vs. Multi-Turn / Agentic. Coding tasks differ considerably in the level of interaction they require. Some tasks, such as code translation, can typically be completed in a single step, whereas others naturally require iterative refinement. To capture this distinction, we categorize benchmarks based on their interaction modality:

Table 5. Classification of benchmarks based on Interaction Paradigm

Paradigm	# Datasets	# Studies	References
Single-Turn	128	114	[4, 5, 7, 11–16, 20, 22, 23, 25–27, 29, 31, 33–35, 40–43, 45, 48, 53, 55–57, 59, 60, 63, 65, 71–73, 75, 76, 82, 84, 87, 89, 91, 93, 94, 97–102, 104, 107, 109–112, 114–118, 120–122, 128, 130, 132–134, 138–142, 145, 146, 150, 152, 154, 158, 159, 161–163, 166, 167, 172, 178–180, 183, 185, 186, 188–192, 194, 196, 198, 200, 202–204, 207, 212, 215–217, 220, 224, 225]
Multi-Turn	28	28	[8, 24, 38, 58, 62, 68, 73, 77, 86, 90, 95, 113, 124, 126, 129, 131, 156, 160, 171, 174–176, 181, 187, 193, 205, 206, 222]
Unique Total	156	142	-

Table 6. Distribution of benchmarks across various Application Domains

Domain	# Datasets	# Studies	References
Algorithm	78	70	[4, 13, 14, 20, 22, 23, 26, 29, 31, 34, 35, 41, 42, 53, 55, 57–60, 63, 65, 72, 75, 76, 82, 89, 97–100, 107, 110–112, 114, 117, 121, 122, 128–131, 133, 139–141, 146, 152, 156, 159, 163, 167, 172, 174, 176, 178, 181, 183, 185, 188, 190–192, 194, 196, 207, 212, 215–217]
Data Science	19	15	[24, 25, 33, 68, 73, 84, 87, 91, 102, 179, 180, 186, 200, 203, 204]
Web Development	13	13	[86, 93, 95, 109, 113, 124, 134, 142, 145, 160, 187, 193, 220]
Security	9	9	[5, 12, 27, 43, 101, 118, 150, 161, 198]
Composite and Niche Domains	36	34	[7, 8, 11, 15, 16, 38, 40, 45, 48, 56, 62, 71, 77, 90, 94, 104, 115, 116, 120, 126, 132, 138, 154, 158, 162, 166, 171, 175, 189, 202, 205, 206, 222, 224, 225]
Unique Total	156	142	-

- **Single-Turn:** These benchmarks treat the model as a stateless function mapping an input prompt (e.g., a function signature) to a static output (e.g., the code body). The model does not receive feedback or have the opportunity to refine its solution based on compiler errors or execution results.
- **Multi-Turn / Agentic:** This category evaluates the ability of a model to engage in iterative problem-solving. In these benchmarks, the model functions as an agent that can interact with an environment (e.g., receiving feedback from test execution, reading file contents, or requesting clarification) over multiple turns to converge on a correct solution.

Table 5 presents the distribution of the benchmarks between these interaction modalities.

4.1.5 Application Domain. Coding tasks are far from uniform; the challenges involved in implementing a low-level sorting algorithm differ fundamentally from those in designing a web API or analyzing a dataset. Consequently, we classify the surveyed benchmarks into five distinct application domains:

- **Algorithm:** This domain serves as the foundational layer of evaluation, focusing on core computer science concepts such as data structures, sorting, and dynamic programming. Problems in this category [14, 29] are typically language-agnostic and self-contained, testing pure algorithmic logic without reliance on external software dependencies.
- **Data Science:** These benchmarks evaluate proficiency with the modern data analysis stack, including statistical analysis, model training, and visualization. Unlike algorithmic tasks, success here requires mastering third-party APIs such as Pandas and Scikit-learn to construct valid data pipelines [33, 87, 204].
- **Web Development:** Targeting the full-stack web development, this domain covers tasks ranging from generating HTML/CSS interfaces to implementing RESTful APIs with frameworks like Django. Benchmarks such as WebBench [187] require an understanding of client-server interactions, database schemas, and occasionally multi-modal inputs for UI generation.
- **Security:** This critical domain evaluates a model’s ability to ensure code safety by identifying vulnerabilities and generating patches. The benchmarks ground their evaluation in the Common Weakness Enumeration (CWE) framework, testing awareness of specific exploits such as SQL injection [5, 161].

- **Composite and Niche Domains:** This category captures specialized or broad-spectrum tasks. It includes Composite benchmarks such as FullStack Bench that evaluate comprehensive full-stack programming capabilities rather than isolated skills that simulate real-world repositories with multiple mixed domains [115], as well as Niche verticals such as mobile development [38].

As illustrated in Table 6, the landscape is heavily skewed towards general-purpose algorithmic tasks. The Algorithm domain is by far the largest, comprising exactly 50% of the surveyed datasets (78 out of 156). Composite and niche benchmarks form the second-largest group (36 datasets), reflecting a shift towards realistic, multi-domain software engineering tasks. Specialized domains such as Data Science (19), Web Development (14), and Security (9) represent smaller but critical clusters, highlighting the growing need for LLMs to master domain-specific libraries and security protocols.

4.1.6 Publication Venues. The detailed distribution of the publishing venues is presented in Table 7. 156 benchmark datasets designed to evaluate LLMs in software engineering, published in a diverse set of academic venues. Of these, 89 datasets (57%) were introduced at conferences and symposiums, reflecting the community’s strong emphasis on timely dissemination and empirical evaluation. An additional 57 datasets (approximately 36.5%) were published as preprints, via arXiv, allowing rapid publication cycles and early adoption within the research community. The remaining 10 benchmarks (6.5%) appeared in peer-reviewed journals, indicating more mature and comprehensive contributions.

The venues span a wide spectrum of disciplines, underscoring the interdisciplinary nature of LLM research in code intelligence. A substantial portion of datasets have been published at premier machine learning conferences, including NeurIPS, ICLR, and ICML, reflecting the increasing integration of LLM-based tools and benchmarks into the core AI landscape [13, 60, 77, 154]. Simultaneously, many contributions have emerged from software engineering-focused venues such as ICSE, FSE, ASE, and TOSEM, highlighting the practical and domain-specific relevance of these benchmarks [48, 132, 181, 183]. Natural language processing venues, including ACL, EMNLP, and NAACL, have also played a critical role, particularly for benchmarks focused on tasks that blend code and language [38, 191, 200]. Finally, the large number of benchmarks distributed through arXiv in recent years, more than 26 in 2025 alone, illustrates the growing commitment of the field to open science and agile resource sharing before formal publication. Taken together, this diverse distribution across AI, SE, and NLP communities demonstrates both the technical breadth and the practical impact of LLM benchmarks in the code intelligence domain.

4.1.7 Programming Languages. The detailed distribution of programming languages can be found in Table 8. Python emerges as the most prevalent programming language across LLM benchmark datasets in code intelligence, featured in 120 (77%) of the benchmark datasets as the only language or in conjunction with others. Java and C++ follow as the next most commonly represented languages, reflecting their widespread use in both academic and industrial software development. However, in recent years, there has been a marked shift toward multilingual benchmarks, with several datasets now encompassing more than 10 or more programming languages [13, 22, 23]. This expansion underscores a growing emphasis on evaluating the multilingual and cross-lingual capabilities of large language models, aligning benchmark design more closely with the linguistic and syntactic diversity encountered in real-world code intelligence tasks. However, many low-resource or domain-specific programming languages, such as Haskell, Rust, or COBOL, remain underrepresented. This gap highlights the need for future benchmarks to more comprehensively reflect the heterogeneous nature of software ecosystems and to facilitate evaluation in low-resource settings.

Our analysis indicates that the selection of programming languages for benchmarks is not arbitrary but correlates strongly with the specific nature of the coding task. Tasks focused on constructive capabilities and algorithmic reasoning, such as Code Generation and Data Science agents, are overwhelmingly dominated by

Table 7. Distribution of LLM benchmark datasets based on conference and journal venues

Conference Venues	# Datasets	Reference	Journal Venues	# Datasets	Reference
ICLR	14	[13, 23, 58, 71, 72, 77, 116, 128, 131, 179, 193, 225]	IEEE TSE	5	[22, 22, 104, 152, 166]
NeurIPS	14	[40, 41, 60, 65, 91, 93, 110, 120, 126, 154, 156]	IEEE Access	1	[11]
EMNLP	12	[68, 95, 178, 188, 191, 203]	TOSEM	1	[181]
ACL	8	[4, 38, 82, 90, 94, 163, 190, 200]	Science	1	[100]
ICSE	8	[26, 42, 45, 73, 75, 132, 202]	EAAI	1	[59]
ASE	5	[20, 27, 48, 76, 150]	TMLR	1	[117]
COLM	3	[111, 159, 185]	arXiv	57	[14, 16, 24, 25, 29, 33, 34, 86, 97, 109, 115, 121, 138, 186, 187, 194, 216, 224] [7, 8, 12, 15, 43, 55, 62, 101, 114, 129, 134, 141, 142, 158, 162, 174, 206, 212] [5, 56, 63, 98, 99, 102, 107, 113, 133, 161, 167, 171, 175, 176, 189, 198, 205, 222]
ICML	4	[31, 53, 87, 124]			
NAACL	6	[57, 84, 146, 172, 180, 196]			
ISSTA	2	[192, 207]			
IJCAI	2	[204, 204]			
AAAI	2	[35, 220]			
FSE	1	[183]			
ICST	1	[89]			
KDD	1	[217]			
HPDC	1	[130]			
AITEST	1	[139]			
ISSRE	1	[112]			
LREC-COLING	1	[140]			
MSR	1	[118]			
PACMPL	1	[145]			
Overall	89		Overall	67	

Python, reflecting its dominant presence. In contrast, maintenance-centric tasks (e.g., Automated Program Repair, Fault Localization, and Vulnerability Detection) demonstrate a heavy reliance on static, compiled languages like C, C++, and Java, aligning with industrial priorities. Furthermore, benchmarks targeting Web Development drive the inclusion of JavaScript, underscoring the necessity of these languages when evaluating models in multi-file, web-centric repository environments.

4.1.8 Data Sizes. The detailed distribution of the sizes of the datasets can be found in Figure 6. Due to the significant variation in size, ranging from just 14 to 189 million samples, we used a log-scale ($\log_{10}$) to visualize the distribution. Detailed size data for each benchmark are available in our replication package⁵. The 156 code intelligence LLM benchmark datasets exhibit a highly skewed size distribution, ranging from just 14 to 189 million samples. We observe a distinct peak in the interval $[10^{2.14}, 10^{2.98}]$ (approximately 138 to 955 samples), which contains 60 datasets. Many widely used benchmarks fall within this range, including HumanEval with 164 problems [29] and Defects4J-NL2Fix with 283 buggy programs [45]. The second-largest cluster (46 datasets) lies in the $[10^{2.98}, 10^{3.82}]$ interval (approx. 955 to 6,600 samples). This range supports statistically significant comparisons while remaining computationally feasible for complex evaluations. Examples include MBPP [14] (974 tasks) and SWE-Bench [77] (2,294 repository-level issues). It should be noted that the average size of the datasets is on the order of 1631641 samples, while the median is around 759, reflecting a heavy right-tail in the distribution. The standard deviation is also large (on the order of 15987155), which highlights high variability. This wide distribution supports diverse evaluation objectives: smaller datasets allow fine-grained, interpretable assessment, and human evaluation, while larger datasets enable fine-tuning code-specific models, benchmarking at scale, and facilitate comparisons across models and configurations. Furthermore, some benchmarks report multiple subsets of varying sizes to reflect different difficulty levels or tasks [77, 82, 179], offering flexible evaluation protocols. Finally, this heterogeneity in the size of the datasets underscores the importance of targeted and large-scale evaluations to fully assess LLM performance in code intelligence contexts.

4.1.9 Evaluated LLMs. The distribution of LLMs can be found in Table 9. Across the 156 benchmark datasets analyzed in this survey, we observe significant diversity in the LLMs used for the evaluation. In total, more than

⁵https://github.com/mohammad-abdollahi/survey_llm_benchmark_dataset

Table 8. Distribution of programming languages across benchmark datasets

Programming Language	#Studies	#Datasets	References
Python	106	120	[4, 7, 8, 11, 13, 14, 16, 22–26, 26, 27, 29, 31, 33–35, 40–42, 48, 53, 55, 57–60, 62, 63, 65, 68, 71–73, 76, 77, 82, 84, 86, 87, 90, 93–95, 97, 99, 100, 102, 104, 107, 109–111, 113–116, 120, 122, 126, 128, 129, 131, 133, 134, 138, 140, 141, 145, 146, 154, 156, 160, 162, 163, 166, 167, 171, 172, 172, 174–176, 178–181, 183, 185, 186, 188–191, 194, 196, 200, 202–204, 215–217, 224, 225]
Java	51	59	[4, 13, 20, 22, 23, 27, 40, 45, 48, 62, 65, 75, 82, 89, 90, 98, 101, 107, 109, 112, 114–116, 120, 121, 128, 139, 140, 152, 154, 156, 158, 159, 161–163, 174, 183, 189–191, 196, 202, 205–207, 212, 217, 220]
Ada	2	5	[190, 191]
Assembly	2	4	[154, 190]
Bash	3	5	[27, 115, 154]
BBC Basic	1	4	[191]
C	13	19	[5, 12, 22, 23, 43, 65, 82, 90, 98, 109, 114, 118, 128, 130, 154, 156, 161, 174, 190, 191, 198, 205, 222]
C#	18	26	[13, 22, 23, 27, 40, 76, 82, 107, 114, 115, 120, 128, 140, 154, 156, 174, 190, 191]
C++	29	37	[5, 12, 13, 15, 22, 23, 27, 65, 76, 82, 90, 107, 109, 114, 115, 118, 128, 130, 154, 161–163, 174, 189–192, 196, 198, 205, 217]
Cobol	2	5	[190, 191]
CSS	2	2	[128, 187]
D	5	9	[22, 82, 115, 190, 191]
Dafny	1	1	[117]
Dart	4	6	[23, 154, 156, 190]
Delphi	3	8	[154, 190, 191]
Elisp	1	1	[23]
Elixir	4	9	[23, 154, 156, 191]
Erlang	3	6	[23, 156, 191]
F#	4	7	[23, 114, 190, 191]
Fortran	3	6	[23, 190, 191]
Go	20	28	[13, 22, 23, 27, 62, 82, 100, 107, 109, 114, 115, 120, 140, 154, 156, 189–191, 205, 217]
Groovy	2	2	[23, 190]
Haskell	5	8	[22, 23, 82, 154, 190]
HTML	7	7	[23, 27, 109, 114, 115, 128, 187]
J	1	4	[191]
Javascript	23	27	[13, 22, 23, 62, 76, 82, 90, 107, 109, 114, 115, 120, 124, 128, 140, 154, 156, 187, 190, 193, 205, 217]
Julia	5	9	[22, 23, 114, 190, 191]
Kotlin	8	13	[13, 22, 23, 82, 140, 154, 156, 190]
Lua	5	9	[22, 23, 100, 190, 191]
Matlab	3	6	[128, 190, 191]
MySQL	1	1	[156]
Objective-C	2	4	[154, 190]
OCAML	2	5	[82, 191]
Pascal	4	7	[23, 82, 114, 191]
Perl	8	16	[13, 22, 23, 82, 140, 154, 190, 191]
PHP	16	24	[13, 22, 23, 27, 82, 100, 107, 109, 114, 115, 128, 140, 154, 156, 190, 191]
R	8	14	[22, 23, 114, 115, 128, 154, 180, 190, 191]
Racket	2	2	[23, 156]
Ruby	15	20	[13, 22, 23, 82, 100, 107, 109, 114, 115, 120, 128, 140, 154, 156, 190]
Rust	12	18	[22, 23, 82, 100, 114, 115, 134, 154, 156, 190, 191, 205]
SAS	1	1	[190]
Scala	13	20	[13, 22, 23, 82, 100, 114, 115, 140, 154, 156, 190, 191]
Shell	2	4	[23, 154]
SQL	4	6	[91, 115, 128, 154]
Swift	9	17	[13, 22, 23, 114, 140, 154, 156, 190, 191]
TypeScript	15	20	[13, 22, 23, 40, 62, 100, 107, 109, 115, 124, 128, 140, 154, 156, 205]
Visual Basic	3	6	[23, 190, 191]
Solidity	3	3	[109, 142, 161]

100 different LLMs were used, reflecting the rapid developments in the field and the growing complexity of evaluation requirements.

From the table, we can see that 85% of the reviewed articles used models from the Open-AI GPT family, including GPT-3 [18], GPT-3.5-turbo, and GPT-4 [1], which have become widely adopted due to their strong performance in both natural language and code-related tasks. In particular, GPT-3.5-Turbo and GPT-4 were the most frequently used models in the reviewed articles, appearing more than 100 times, respectively [65, 95, 128, 225]. These models were often used as baselines or upper bounds for performance comparisons. We can also observe that 58% of benchmarks evaluated code-specific LLMs, models fine-tuned on source code corpora, such as CodeLlama [149], StarCoder [96], CodeGen [131], Codex [29], CodeT5 [173], and CodeQwen [69]. These models demonstrate strong performance on code generation, completion, and repair tasks and are increasingly central in code intelligence studies [93, 114, 115, 196].

The LLaMA family of models appeared in 46% of the reviewed articles, including variants such as LLaMA 2 [164], LLaMA 3 [52], and CodeLlama [149]. These models are often preferred in open source evaluations due to their accessibility and strong performance when fine-tuned for code tasks [15, 34, 110, 194].

A clear distinction is observed between closed-source and open-source models. Closed-source models, such as GPT-3.5-Turbo, GPT-4 [1], and Claude 3 Sonnet, are the most commonly used proprietary models and consistently demonstrate strong performance in terms of accuracy, robustness, and task generalization. However, their use is limited by the lack of transparency and reproducibility. In contrast, open-source models, notably CodeLLaMA [149], Mistral [74], and LLaMA 3.1, are widely adopted due to their accessibility, customizability, and reproducibility, making them particularly well suited for community-driven evaluations and iterative development of benchmark datasets.

4.1.10 Benchmark Adoption and Community Impact. Although there has been a large number of benchmarks emerging, it is subjective to conclude that the effects on the community have been evenly distributed. To objectively assess this impact, we utilized the number of citations as a proxy for community adoption and influence. Figure 5 illustrates the distribution of citations across the surveyed papers. Based on our analysis on 156 benchmarks that were surveyed, we observe a “power law” distribution in benchmark adoption, where a small number of datasets serve as the primary evaluation standards, while the majority form a long tail of specialized or emerging datasets.

Widely Adopted Benchmarks: The landscape is dominated by a few foundational benchmarks that have become the standard for comparison. HumanEval [29] stands out as the primary baseline with over 7,100 citations, followed by MBPP and MathQA-Python [14] with roughly 2,600 citations each. CodeContests [100] and MTPB [131] also fall into this top tier (>1,500 citations). Their high adoption is largely due to their early release (2021-2022) and their role in establishing the initial evaluation protocols for Code LLMs.

Domain Leaders and Rising Stars: A second tier of high-impact benchmarks (roughly 200–1,500 citations) has emerged to address specific gaps. SWE-bench [77] (1,304 citations) has rapidly established itself as the industry standard for agent-based software engineering and issue resolution. In the domain of competitive programming and algorithmic complexity, APPS [60] (914 citations) remains a key reference. Notably, newer benchmarks from 2024 are quickly gaining traction; for example, LiveCodeBench [72] has already garnered over 800 citations by addressing data contamination, and DS-1000 [87] (390 citations) has become the go-to for data science tasks.

The Long Tail: The majority of the 156 surveyed datasets fall into the “long tail” with fewer than 100 citations (e.g., ConDefects, ProjectEval). This does not necessarily imply lower quality; rather, these benchmarks often target highly specific tasks, such as vulnerability detection [5], mobile development [38], or specific languages like Rust, that are critical for specialized applications but have a narrower user base than general-purpose Python generation.

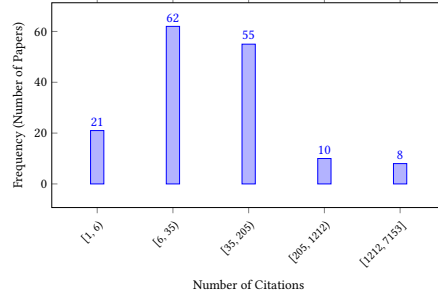
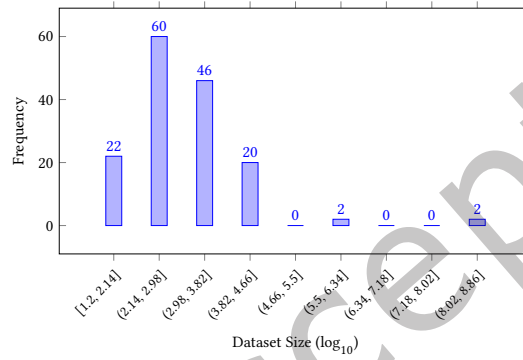


Fig. 5. Distribution of citations for the papers introducing LLM benchmark datasets

Fig. 6. Distribution of LLM benchmark dataset sizes (scaled using $\log_{10}$)**Answer to RQ1:**

- (1) The majority of the selected papers were published within the last three years (2023–2025), indicating a clear upward trend in research activity since 2023.
- (2) Most studies concentrate on code generation as the primary evaluation task.
- (3) A significant portion of these papers appear in leading machine learning venues, such as ICML, NeurIPS, and EMNLP.
- (4) Python emerges as the dominant programming language represented in the datasets.
- (5) The dataset sizes exhibit substantial variability, ranging from as few as 14 samples to as many as 189 million.
- (6) The OpenAI GPT family of models (e.g., GPT-3.5, GPT-4o) are the most frequently used LLMs in evaluation experiments.

4.2 RQ2: Characteristics and Quality of Benchmark Datasets

In this section, we examine the structure, composition, and quality controls of LLM benchmark datasets. We analyze their data sources, key characteristics, and the measures taken to ensure the relevance and representativeness of real-world scenarios.

4.2.1 What are the sources of datasets? The construction of LLM benchmark datasets in code intelligence is fundamentally shaped by the source of the underlying data, which influences the realism, diversity, and

Table 9. Distribution of LLMs in the subject studies

Category	#Studies	Models	References
OpenAI GPT-family	117	o1-preview, o1-mini, GPT-4, GPT-4o, GPT-4o-mini, GPT-3, GPT-3.5-Turbo, GPT-4-Turbo, Text-DaVinci-003, Code-DaVinci-002, Code-DaVinci-Edit-001, Codex-DaVinci-002, Codex-DaVinci-001	[5, 7, 8, 11, 14, 16, 20, 23, 24, 26, 27, 31, 33–35, 38, 40, 42, 43, 45, 48, 53, 55–60, 62, 65, 68, 71–73, 76, 77, 84, 86, 87, 90, 91, 93–95, 97–99, 101, 102, 104, 107, 109–115, 117, 120–122, 124, 126, 128–131, 133, 134, 138–142, 146, 154, 156, 158–163, 166, 167, 171, 172, 174–176, 179–181, 185–193, 196, 198, 202, 204–207, 212, 215, 216, 220, 222, 224, 225]
Meta LLaMA-family	80	LLaMA 3, LLaMA 3.1, LLaMA 2, LLaMA, CodeLLaMA, TinyLLaMA	[7, 15, 16, 23, 24, 26, 27, 31, 33–35, 38, 41, 43, 48, 53, 55, 57, 63, 65, 71, 72, 82, 86, 90, 93, 94, 97–99, 102, 107, 109–111, 113–115, 117, 121, 122, 124, 128, 130, 134, 138, 141, 142, 154, 158–163, 166, 167, 172, 174–176, 179–181, 185–190, 192, 194, 196, 207, 212, 215, 216, 220, 224, 225]
Code-centric Models	63	StarCoder, StarCoder2, CodeGen, InCoder, CodeT5, WizardCoder, CodeGemma, DSCoder, OpenCoder, CodeGeeX, CodeGPT, CodeBERT, Magicoder-S-DS, PLBART, Magicoder, StarCoder-Base, GraniteCoder, SantaCoder, CodeT5+, PolyCoder, TransCoder, DSCoder V2, CodeGen2, CodeClippy, GraphCodeBERT, CodeParrot, PanguCoder, OctoCoder, StableCode, PyCodeGPT, PalmCoder, StarChat	[4, 20, 22, 26, 31, 34, 40–42, 48, 53, 55, 57, 65, 72, 75, 76, 82, 84, 87, 91, 93, 94, 97, 99, 110, 111, 114–116, 120, 121, 129–132, 138, 140–142, 152, 156, 159, 162, 172, 176, 178, 185, 188, 190–192, 194, 196, 200, 202–204, 207, 215–217, 225]
Anthropic Claude-family	46	Claude 3.5 Sonnet, Claude 3 Opus, Claude 3 Haiku, Claude 3 Sonnet, Claude 2, Claude 1, Claude 3.5 Haiku, Claude Instant	[5, 24, 31, 33, 35, 57, 65, 68, 72, 77, 86, 91, 98, 99, 109, 114, 115, 117, 122, 124, 126, 128, 129, 134, 138, 141, 146, 154, 158, 167, 171, 174–176, 179, 180, 185, 187, 189, 192, 193, 205, 207, 215, 216, 225]
Mistral-family	43	Mistral, Mixtral, CodeStral	[5, 7, 15, 16, 23, 26, 31, 33, 34, 43, 53, 56, 57, 62, 65, 68, 71, 72, 84, 99, 109, 110, 114, 122, 126, 128, 138, 141, 154, 158, 159, 163, 174, 179, 180, 185, 187–189, 194, 215, 216, 225]
Alibaba Qwen-family	37	CodeQwen 1.5, Qwen 2.5, Qwen 2, Qwen 2.5 Coder, Qwen 1.5, Qwen-Coder 2.5, Qwen1	[5, 23, 31, 41, 43, 57, 68, 86, 93, 98, 99, 102, 111, 114, 115, 122, 128, 138, 142, 158, 161, 167, 175, 176, 179, 181, 185, 187–189, 194, 196, 205, 215, 216, 224, 225]
DeepSeek-family	37	DeepSeek, DeepSeek V2, DeepSeek V2.5, DeepSeek-Coder V2, DeepSeekCoder 2.5, DeepSeek R1	[5, 31, 34, 43, 57, 63, 68, 86, 98, 99, 102, 115, 124, 128, 134, 138, 141, 142, 145, 158, 161, 162, 167, 171, 174–176, 180, 181, 185, 187, 189, 196, 205, 206, 224, 225]
Google Gemini-family	25	Gemini Flash, Gemini Pro	[31, 43, 57, 62, 72, 86, 99, 109, 113, 133, 141, 154, 158, 167, 172, 175, 176, 179, 180, 185, 187, 189, 215]
Microsoft Phi-family	18	Phi-2, Phi-3, Phi-1.5, Phi-4, Phi-4 Mini, Phi-3 Mini	[31, 34, 53, 57, 63, 99, 102, 113, 128, 141, 161, 167, 179, 185, 188, 189, 205, 224]
others	53	Vicuna, GPT-Neo, Gemma, Gemma 2, GPT-J, YiCoder, Yi, ChatGLM, Phind, PaLM 2, Yi 1.5, Phind V2, OpenCodeInterpreter-DS, Lynx, Alpaca, QwQ, AlphaCode, PaLM, RoBERTa, BART, StableLM, Doubao-1.5-pro, WizardLM, Zephyr, Tulu-2, Vicuna 1.5, SwellLLaMA, Bard, GLM-4	[14, 22, 23, 25–27, 29, 34, 42, 53, 55, 56, 60, 65, 71–73, 77, 87, 91, 93, 95, 98, 100, 109, 110, 114–116, 120, 128, 131, 132, 138, 154, 156, 160, 172, 178, 179, 185, 187, 188, 192, 194, 200, 203–205, 215, 217, 220, 225]
Unique Total	137	-	-

generalizability of the benchmarks. Table 10 illustrates the distribution of data sources for the creation of

benchmark datasets. In our review of 142 studies, we observe that these sources fall into four categories: open source repositories and GitHub, manually curated data, online programming platforms, and existing benchmarks. It should be noted that some datasets have been created by combining multiple source types to enhance their coverage.

A substantial portion of datasets 66 (46%) derive their data from GitHub, reflecting its role as a central repository of real-world, multi-domain software projects. These datasets include DevEval [94], SWE-Bench [77], RepoEval [104], and ExecRepoBench [194], among others, and leverage repositories, issues, pull requests, and version histories to simulate authentic code intelligence contexts. We can also see that 37 (26%) of the datasets are created manually entirely from scratch. Benchmarks such as HumanEval [29], MBPP [14], LBPP [122], and Api-Bank [95] fall into this category, emphasizing clean task design, controlled evaluation environments, or novel formats.

Online programming platforms such as LeetCode, AtCoder, and CodeForces contribute to a smaller but notable share of datasets, consisting of 35 (24.6%) datasets. 15 (10.5%) of the benchmarks [55, 156, 190] rely on LeetCode as their primary source, while Codeforces.com and AtCoder appear in roughly 5 (3.5%) of the datasets [72, 82, 97].

In addition, 32 (22.5%) datasets are derived from existing LLM benchmarks, either by augmentation, translation, or integration. Examples include HumanEval+ [110], SWE-Bench+ [8], and CodeJudge-Eval [215], which expand based on well-established benchmarks such as HumanEval [29], SWE-Bench [77], and APPS [60] to create more comprehensive or domain-specific evaluations. Furthermore, some benchmarks, such as NL2Fix [45], reuse older datasets like Defects4J [80] to support LLM evaluation.

Other notable sources include educational platforms like GeeksForGeeks.com, Kaggle, Rosetta Code, W3School, Q&A forums like StackOverflow, NVD, and specialized programming sources such as *Dive into Deep Learning Platform*⁶.

In general, the diverse sourcing strategies across the benchmarks reflect a balance between controlled evaluation through manual creation and real-world complexity through open-source and platform-derived data. The increasing reliance on realistic repository-scale data from platforms such as GitHub suggests a shift toward evaluating LLMs in practical code intelligence scenarios, while continued use of curated or translated datasets allows for targeted probing of model capabilities in multilingual, domain-specific, or long-context settings.

4.2.2 How are the benchmark datasets structured and organized? One of the most important aspects of LLM benchmark datasets is their structure and practical usability. A well-defined and accessible structure is vital in order to support future research. This will enable the application of more sophisticated theoretical methods and facilitate a consistent evaluation by using the tests provided or canonical solutions of the benchmarking dataset.

Many of the datasets have a problem-centric structure, meaning that they are designed for evaluating LLMs by providing a problem description for LLM and having unit tests to test the provided solution by LLM, or have canonical solutions that will try to compare the LLM-generated solution with the canonical solution. Datasets such as HumanEval, MBPP, Math-QA Python, and APPS [14, 29, 60] all follow this structure. The updated variants, such as HumanEval+ or LBPP [110, 122], have kept the same structure, but have tried to increase the test coverage and handle minor problems in canonical solutions, while datasets such as HumanEval-X and MBXP [13] have extended these problems to multiple programming languages. The other pattern that we can see is for the datasets designed for code translation [22, 174, 191]. These datasets are organized in a way that includes parallel or equivalent codes in different programming languages, and even sometimes include unit tests to check runability and semantics of the generated codes. Furthermore, the G-TransEval [76] dataset has gone further and has grouped tasks into different types and complexity levels to ensure the LLM's ability in code translation.

Datasets focused on bug detection and debugging [158, 163] organized data as buggy code snippets paired with explanations, fixes, golden patches, and unit tests. These datasets often classify the types of bugs or stages of the debugging process.

⁶<https://github.com/d2l-ai/d2l-zh/blob/master/LICENSE>

Table 10. Detailed distribution of benchmark sources

Source	Detailed Sources	# Studies	References
GitHub	-	66	[7, 8, 11, 12, 15, 16, 25, 27, 38, 40, 45, 48, 57, 62, 65, 68, 71, 77, 84, 86, 89, 90, 93–95, 97, 99, 101, 102, 109, 114–118, 120, 126, 130, 134, 138, 142, 145, 152, 159, 161–163, 175, 179, 181, 183, 185–187, 190, 193, 194, 198, 200, 202, 205, 206, 212, 215, 220, 224]
Manually Created	-	37	[11, 14, 20, 23, 27, 29, 33, 34, 42, 43, 60, 73, 75, 86, 90, 91, 95, 104, 107, 112, 115, 120, 122, 129, 131, 132, 140, 160, 178, 179, 188, 192, 203, 204, 206, 217, 225]
Existing Datasets	HumanEval	11	[13, 22, 31, 110, 111, 128, 141, 146, 167, 176, 181]
	MBPP	7	[13, 22, 110, 111, 141, 171, 176]
	Defects4J	4	[94, 158, 206, 207]
	APPS	4	[97, 141, 176, 215]
	ClassEval	2	[26, 216]
	Code-Contest	2	[97, 141]
	CodeNet	2	[35, 128]
	CodeSearchNet	2	[120, 176]
	PrimeVul	2	[5, 198]
	MBPP+	2	[63, 216]
	Others	16	[13, 31, 97, 113, 117, 128, 158, 171, 174, 176, 179, 189, 189, 196, 203, 216]
Coding Websites	LeetCode	15	[41, 55, 63, 65, 72, 97, 120, 128, 156, 163, 167, 172, 179, 188, 190]
	AtCoder	5	[60, 72, 100, 179, 183]
	CodeForces	5	[60, 72, 82, 97, 100]
	CodeChef	3	[59, 97, 100]
	CodeWars	2	[60, 97]
	Kattis	2	[60, 97]
	Aizu	1	[100]
	HackerEarth	1	[100]
	HackerRank	1	[97]
StackOverflow	-	5	[87, 139, 154, 178, 220]
GeekForGeeks	-	4	[76, 97, 128, 188]
Kaggle	-	5	[24, 68, 91, 179, 200]
National Vulnerability Database	-	3	[5, 12, 118]
W3School	-	2	[128, 139]
CVE	-	2	[5, 161]
Common Crawl	-	1	[128]
Frama-C-Problems	-	1	[121]
Jira	-	1	[94]
JML	-	1	[121]
Dive into Deep Learning Platform	-	1	[191]
Rosetta Code	-	1	[191]
Socrata	-	1	[179]
SpecGen	-	1	[121]
SVN	-	1	[94]
Upwork	-	1	[124]
Unique Total	-	142	-

Large-scale and multitask datasets, such as CodeXGlue and LongCodeArena [16, 120], which are designed to evaluate multiple tasks, are designed in a modular way to be able to evaluate multiple tasks, from code generation to code summarization, and usually consist of metadata and the required structure to evaluate each code intelligence task.

Data-science focused benchmarks [25, 87, 200] try to reflect the real and interactive structure of the data-science environment. they have organized in a way to be in a close resemblance to the real world, such as having multiple jupyter notebook cells with context, having grading and evaluation pipelines such as kaggle medals, or analyzing multiple data science and deep learning libraries and other complex data science components in their structure.

Some benchmarks, such as RepoEval, ComplexCodeEval, and RepoBench, are structured in a way to reflect multi-file, repository-level, or long-term context relationships, and more complex dependency relations between different components in code intelligence tasks. These benchmarks include a full snapshot of the repository, issue metadata, multiple contexts, and test suites required to evaluate such dependencies.

Finally, some datasets are organized to assess a specific area in code intelligence. For example, SWE-BENCH MULTIMODAL [193] includes not only code and text, but also images within issue descriptions and test cases to evaluate the multimodal nature of code intelligence tasks. The other examples are Mobile-Bench and EFFIBENCH [38, 65], which focus on the development of mobile applications or efficiency-critical requirements, consist of domain-specific metadata, and have provided the context and environment required to handle such tasks.

4.2.3 What are the quality controls applied in the data collection process? Ensuring the quality of the data collected is a key step in the dataset creation process. Benchmark datasets should incorporate powerful quality assurance procedures to maintain high standards and prevent issues such as inconsistency, incorrect labels, poor coverage, or data contamination. Table 11 presents a detailed categorization of the quality control approaches employed by each surveyed benchmark. It should be noted that a large proportion of 43% of datasets (67 datasets) did not apply quality controls to their data collection pipeline. We can categorize the quality control measures described in the reviewed articles into the following groups.

- **Manual Inspection and Expert Review:** Human inspection and expert knowledge remain among the most reliable methods for validating results and processes in benchmark creation. Although minor human errors can occur, this approach is particularly well-suited to the technical demands of code intelligence. As shown in Table 11, 32% of the datasets employed this method. Quality assurance in this category includes a detailed manual review by authors or experts in the domain of the entire data set [112, 122] or selected subsets [97, 191]. Some projects even used external experts specifically for validation [160].
- **Multi-Annotator and Peer Verification:** Several datasets (9%) employed independent annotators, different from those who created the initial data, to validate the entries. This approach often involved volunteers or professional engineers tasked with verifying and cross-checking the collected data to ensure its accuracy and consistency. e.g., in the FullStackBench [115], after completing their initial annotations, each annotator exchanges their work with another annotator for cross-refinement, in order to reduce subjective bias and errors. Similarly, in MCEval [23], each sample is independently annotated by at least two annotators to mitigate subjectivity, with any discrepancies resolved through consensus discussions or adjudication by a senior annotator.
- **Automated Filtering:** Automated techniques were applied in 8% of the benchmarks, including removing duplicates, non-compiling or redundant code, and other problematic instances. Automated filters are particularly essential for large-scale datasets where manual review of all entries is impractical. For example, CodeRepoQA [62], after collecting approximately 636,000 GitHub issues, we applied automated filtering criteria (i.e., including content length, duplication, external links, event count, robot detection, and participant count) to ensure data quality, resulting in 585,687 high-quality, code-related QA pairs. SecVulEval [5] applies automated filtering criteria that include removing non-C files, excluding multi-commit fixes, eliminating commits with no function changes, and using heuristics to discard refactoring or reformatting changes.
- **Hybrid Approach:** A subset of benchmarks (8%) used a combination of manual and automated methods, using large language models to enhance the overall quality assurance process. For example, BigCodeBench [225] employed a multistep process of human curation, LLM-based dry-run evaluation, and final cross-checking. RepoTransBench [174] used an iterative manual correction loop until the repositories were fully compilable. CrossCodeEval [40] applied the LLM-based removal of short, duplicate, or exact match codes before subjecting the data to human annotation.

Our analysis reveals interesting quality control profiles across research communities. Both SE (e.g., ICSE, ASE) and AI (e.g., NeurIPS, ICLR) venues consistently rely on Manual Inspection / Expert Review as a primary mechanism for ensuring dataset validity, indicating a universal preference for human oversight in establishing ground truth. Despite this similarity, the communities diverge in their supplementary quality assurance strategies. SE

Table 11. Quality controls applied in data collection process

Quality Control	# Datasets	# Studies	References
Manual inspection / expert review	50	38	[8, 11, 13, 14, 22, 25, 27, 31, 38, 43, 93, 97, 101, 104, 109, 112, 113, 122, 131, 133, 134, 146, 154, 160, 167, 178, 188, 189, 191–193, 198, 203, 204, 206, 207, 212, 220]
Automated filtering / execution-based checks	15	15	[5, 48, 53, 57, 62, 63, 89, 99, 102, 118, 128, 141, 175, 187, 194]
Multi-annotator / peer verification	12	12	[23, 91, 95, 107, 114, 115, 121, 124, 171, 200, 205, 222]
Hybrid	12	12	[16, 34, 40, 82, 87, 140, 163, 174, 176, 180, 186, 225]
No Quality Control	67	65	[4, 7, 12, 15, 20, 24, 26, 29, 33, 35, 41, 42, 45, 55, 56, 58–60, 65, 68, 71–73, 75–77, 84, 86, 90, 94, 98, 100, 110, 111, 116, 117, 120, 126, 129, 130, 132, 138, 139, 142, 145, 150, 152, 156, 158, 159, 161, 162, 166, 172, 179, 181, 183, 185, 190, 196, 202, 215–217, 224]

Table 12. How datasets ensured real-world representativeness

	# Datasets	# Studies	References
Sourcing from Real-World Data Sources	65	63	[4, 8, 12, 15, 26, 27, 33, 35, 41, 45, 48, 55, 59, 62, 65, 72, 76, 77, 82, 86, 89–91, 93, 94, 97, 99–101, 104, 107, 116–118, 120, 124, 126, 128, 134, 139, 142, 150, 152, 156, 159, 161, 162, 167, 174, 175, 179, 180, 183, 186, 190, 193, 202, 203, 205–207, 215, 224]
Emphasis on Realistic Context and Dependencies	15	15	[31, 40, 56–58, 63, 122, 130, 133, 140, 141, 145, 176, 185, 194, 212]
Problem Design to Mirror Developer Workflows	25	25	[7, 16, 25, 60, 68, 71, 84, 87, 102, 111–113, 115, 129, 131, 146, 160, 171, 181, 187–189, 192, 200, 216]
Ensuring Realness through Community and Industrial Validation	13	9	[24, 38, 73, 95, 138, 154, 178, 204, 220]
Coverage and Structural Properties	18	15	[5, 11, 20, 34, 42, 43, 98, 109, 132, 158, 163, 191, 196, 198, 222]

venues tend to complement manual review with Automated filtering / execution-based checks, emphasizing correctness through rigorous, reproducible execution environments. In contrast, AI venues distinguish themselves by adopting Hybrid workflows and Multi-annotator / peer verification, reflecting a tendency to address the scale and subjectivity of large language model outputs through semi-automated protocols.

4.2.4 How do these datasets ensure the quality, relevance, and representativeness of real-world scenarios? In the rapidly growing field of LLM benchmarking, it is essential to ensure that datasets are high-quality, relevant, and truly reflect real-world scenarios. To achieve this, these datasets reported various strategies, such as collecting real-world data, analyzing industrial needs, and applying careful checks. Our categorization of the approaches used by the datasets surveyed to ensure real-world representativeness is shown in Table 12. These approaches help to ensure that the benchmarks closely match real-world code intelligence tasks and provide a meaningful evaluation.

- **Sourcing from Real-World Data Sources, Repositories, and Competitive Programming Platforms:** The majority of datasets (42%) guarantee real-world representativeness by directly sourcing tasks and code from real-world repositories and platforms. For instance, RepoBench [116] focuses on recent, non-forked repositories to prevent data leaks and better represent modern coding practices, while CodeScope [190] sources tasks from active GitHub projects, industry-relevant open-source software, and competitive programming platforms. Furthermore, by using well-known competitive programming platforms, datasets like XCodeEval, TACO, and the LeetCode Contest Benchmark [55, 82, 97] guarantee quality by providing difficult human-authored problems with unit tests. These datasets offer variety and rigor that closely resemble the requirements of solving problems in the real world.
- **Problem Design to Mirror Developer Workflows:** Approximately 16% of datasets model the tasks that developers encounter explicitly in order to achieve realism. APPS [60] mimics human evaluation processes by using real coding sites to solve problems in an unrestricted natural language. The writing or improvement of test suites in meticulously maintained repositories while accurately mimicking actual developer scenarios is the main goal of TestGenEval [71].
- **Emphasis on Realistic Context and Dependencies:** To reflect the complexity of modern coding, 10% of the benchmarks incorporate realistic context and dependencies. ExecRepoBench and CrossCodeEval

[40, 194] assess code completion in scenarios with substantial cross-file dependencies, mirroring actual code intelligence difficulties. TestBench [212] manually chooses functions, commonly used in actual development by sourcing programs from extensive GitHub projects.

- **Ensuring Realness through Community and Industrial Validation:** By choosing content that has been validated by the community or issues that are relevant to the industry, a number of benchmarks emphasize authenticity. To reflect community consensus and usefulness, StackEval [154] selects tasks from StackOverflow questions with upvoted responses. MLE-Bench [24] selects difficult and worthwhile tasks that are typical of modern ML engineering work by methodically screening Kaggle competitions. In addition, Codev-Bench [138] is notable for incorporating user input from an industrial code completion product and adjusting its design to meet the requirements of actual developers.
- **Coverage and Structural Properties:** By imposing structural code properties and including a diverse range of problems, certain benchmarks guarantee coverage and representativeness. For example, ClassEval [42] follows professional software testing standards by producing test suites with high branch and statement coverage. PyBugHive [11] ensures relevance to real-world scenarios by specifically including non-trivial bugs that reflect the typical characteristics of issues encountered in practical software projects. CodeTransOcean [191] demonstrates real-world applicability by including diverse code examples, from basic syntax to complex algorithms and neural network implementation, reflecting a wide range of practical scenarios.

4.2.5 How Do Benchmark Datasets Mitigate Data Contamination? Data contamination, where evaluation data inadvertently leaks into a model’s training corpus, poses a critical threat to the validity of code intelligence benchmarks. When models have “seen” the test cases during training, their performance reflects memorization rather than generalization. To address this, existing benchmarks employ four primary mitigation methodologies (see Table 13):

- **Temporal Isolation & Knowledge Cutoffs:** This strategy ensures that evaluation data strictly post-dates the training period of the LLMs being tested. By selecting data sources or repositories released after a specific “knowledge cutoff” date, these benchmarks prevent the model from having prior exposure to the test set. For example, LiveCodeBench [72] and RepoEval [104] utilize continuous updates or recent repository creation dates to maintain a fresh evaluation set that remains unseen by LLMs.
- **Pre-training Corpus Exclusion:** Rather than relying on dates, this approach algorithmically purges benchmark data that overlaps with common large-scale training corpora. Techniques include N-gram overlap analysis, MinHash deduplication, and exact string matching to identify and remove leaked samples. e.g., BigCodeBench [225] and AL-Bench [162] employ rigorous N-gram analysis to detect and remove contamination. ExecRepoBench [194] explicitly removes samples with significant word overlap from the training corpus to ensure independence.
- **Perturbation and Novelty Construction:** This methodology focuses on creating novel content or modifying existing data so that models must reason rather than rely on memorized solutions. Authors either hand-craft entirely new problems that do not exist on the web or apply semantic perturbations to existing problems to render them unrecognizable to the model while preserving the underlying logic. HumanEval [29] consists of hand-written Python problems specifically designed to be distinct from public GitHub repositories. DS-1000 [87] utilizes extensive surface and semantic perturbations to prevent models from recalling solutions from memory.
- **Environmental Isolation and Private Holdouts:** The most robust mitigation involves restricting access to the data itself. These benchmarks maintain private “holdout” sets that are never released publicly or use proprietary internal data, preventing web crawlers from indexing the answers for future model training.

Table 13. Overview of Data Contamination Mitigation Approaches in Code Intelligence Datasets

Mitigation Methodology	# Datasets	# Studies	References
Temporal Isolation & Knowledge Cutoffs	20	20	[8, 40, 48, 55, 59, 72, 77, 82, 89, 93, 100, 104, 116, 145, 152, 156, 163, 172, 179, 183]
Pre-training Corpus Exclusion	18	17	[13, 25, 57, 60, 71, 97, 98, 114, 117, 128, 132, 150, 162, 194, 200, 204, 225]
Perturbation and Novelty Construction	29	27	[5, 14, 16, 20, 22, 24, 29, 31, 34, 41, 42, 56, 63, 75, 76, 84, 86, 87, 118, 121, 122, 138, 142, 161, 190, 202, 216]
Environmental Isolation and Private Holdouts	3	3	[91, 124, 192]
No Mitigation Measures	86	75	[4, 7, 11–13, 15, 23, 26, 27, 33, 35, 38, 43, 45, 53, 58, 62, 65, 68, 73, 90, 94, 95, 99, 101, 102, 107, 109–113, 115, 120, 126, 129–131, 133, 134, 139–141, 146, 154, 158–160, 166, 167, 171, 174–176, 178, 180, 181, 185–189, 191, 193, 196, 198, 203, 205–207, 212, 215, 217, 220, 222, 224]

SWE-Lancer [124] keeps a substantial portion of its dataset private and conducts evaluations in an offline, sandboxed environment.

Despite these efforts, it is notable that the majority of surveyed datasets (86 out of 156) report **No Mitigation Measures**. This suggests that while awareness of data contamination is growing, it remains an unaddressed limitation in a significant portion of the code intelligence landscape, potentially leading to inflated performance metrics for older or widely utilized benchmarks.

Our cross-dimension analysis reveals that the strategy for mitigating data contamination is heavily dictated by the provenance of the benchmark data. Manually created benchmarks exhibit the highest standard of decontamination, predominantly utilizing manual sanitization to create a clean environment that proactively avoids the risk of memorization. In contrast, benchmarks derived from Coding Websites (e.g., LeetCode) show a strong correlation with Temporal Isolation, leveraging structured publication timestamps to enforce verifiable knowledge cutoffs relative to a model’s training date. Conversely, benchmarks sourced from GitHub and Existing Datasets, which represent the bulk of pre-training corpora, rely largely on reactive measures such as De-duplication or frequently report No Mitigation at all.

Answer to RQ2:

- (1) GitHub serves as the most common source for collecting data to construct benchmark datasets, followed by datasets that are predominantly created through manual curation.
- (2) Most benchmarks are organized around natural language descriptions specifying tasks, combined with unit tests designed to verify the correctness of solutions generated by LLMs.
- (3) The majority of datasets (32%) ensure data quality by incorporating manual inspection of the entire dataset or a representative subset.
- (4) Most of the datasets (49%) aim to enhance real-world relevance by sourcing data and tasks from practical contexts (e.g., GitHub repositories, competitive programming platforms) and designing tasks that closely reflect actual code intelligence challenges.

4.3 RQ3: Evaluation Metrics and Techniques

4.3.1 What evaluation metrics are commonly used to assess LLM performance using these benchmark datasets?

The choice of evaluation metrics in LLM benchmark datasets is highly dependent on the underlying coding task being assessed. Among the 156 reviewed benchmarks, we observe a wide range of metrics tailored to capture correctness, efficiency, robustness, and alignment with human-written code or specifications. Table 14 shows detailed distributions of the metrics in the surveyed papers. In addition, some benchmarks introduce novel task-specific evaluation metrics to assess performance in their unique problem settings. The diversity of metrics underscores the diverse nature of coding tasks and the need for task-specific evaluation protocols.

For code generation and code completion, which remain the most commonly studied tasks, the most widely used metrics are those that assess the correctness and functional accuracy of the generated code. The dominant metric is **Pass@k**, defined as the probability that at least one of the top-k generated codes for a problem passes the unit tests. Pass@k and Pass@1 appear in 51 and 13 papers, respectively [29, 42, 72, 93, 200].

Apart from that, several additional measures are employed across benchmarks to evaluate correctness:

- **Accuracy** is a term frequently used interchangeably with *pass rate*, referring to the percentage of generated code completions that pass all associated test cases. However, in some benchmarks such as *RepoEval* and *StackEval*, it is used in the traditional classification sense to measure prediction correctness.
- **Pass rate** and **success rate** denote the proportion of generated solutions that pass all the test cases [14, 41, 90, 131, 217].
- **Average test case pass ratio** captures the average fraction of test cases passed by the model-generated code and is used in benchmarks such as APPS and CODEC [59, 60].
- **Acceptance rate** measures the proportion of generated solutions considered acceptable based on a predefined rubric, as used in *StackEval* [154].

In addition to the above, benchmarks targeting data science code generation introduce domain-specific metrics that evaluate performance in tasks related to data visualization and analysis, including:

- **Medal Rate:** It is inspired by Kaggle competition scoring and used in *MLE-Bench* [24]. This metric compares agent submissions against the private leaderboard and assigns bronze, silver, or gold medals based on competition-specific thresholds that ensure consistency in performance interpretation across tasks.
- **ML normalized score, chart match score, and table match score**, which are designed to assess the correctness of model outputs in specific subcomponents of data science pipelines, as demonstrated in the *DaCode* benchmark [68].

Besides, various benchmarks focusing on different aspects of code generation, such as multi-file reasoning, efficiency, performance, readability, and maintainability, have introduced specialized evaluation metrics tailored to their specific goals. In the following, we summarize several of these metrics and their corresponding sources:

- **DEP:** Measures the percentage of dependencies correctly generated per method, compared to the total number of dependencies in the canonical solution [42].
- **Efficiency Score (BEYOND):** Computes a run-time percentage-weighted pass score that simultaneously reflects functional correctness and code efficiency [41].
- **Valid Efficiency Score:** Designed to evaluate the efficiency of valid SQL queries, where "valid" refers to predicted SQLs that yield result sets matching the ground-truth. Queries failing to return correct results are considered invalid regardless of their efficiency [91].
- **Differential Performance Score (DPS):** Measures performance shifts relative to a baseline in the EvalPerf benchmark [111].
- **Good Question Rate and Communication Rate:** Used in HumanEvalComm [181] to assess the clarity and quality of user-model interaction.
- **Lines of Code (LOC):** Used as a measure of code verbosity or conciseness [139, 186].
- **Cyclomatic Complexity and Cognitive Complexity:** Used to assess structural and cognitive complexity of generated code [139].
- **Compilation@k:** Measures whether at least one of the top-k generated codes can be compiled successfully [20].
- **Completion Rate:** Indicates the proportion of code generation tasks for which the model returns a complete solution [68].

- **Completion@k**: Evaluates whether at least one of the top- k model outputs constitutes a valid completion [20].
- **Coverage@k**: Measures how well the top- k completions cover different correct strategies or reference solutions [207].
- **Edit Similarity**: Measures the minimal edit operations required for a match between generated and reference code [40, 48, 116, 194].
- **Efficiency@k**: Measures the efficiency of the top- k generated outputs, often focusing on latency, speed, or resource use [130, 216].
- **Execution Rate**: The percentage of generated code that can be executed without runtime errors [129].
- **F1 Score**: Harmonic mean of precision and recall, typically used for classification or structured prediction tasks [16, 104].
- **Library Coverage**: Measures the proportion of used libraries in the generated code that are already pre-installed in the code repository's environment [104].
- **Readability**: As defined in the RACE benchmark [216], readability is assessed based on three key aspects: (1) *Length* — whether the code adheres to standard line or function length limits (e.g., PEP8 guidelines); (2) *Naming Convention* — consistency with camelCase or snake_case naming styles; and (3) *Comment Granularity* — the inclusion of comments at the line or function level.
- **Maintainability Index**: Scores code health based on Halstead Volume, Cyclomatic Complexity, LOC, and comment density. [216].
- **Normalized Efficiency Index**: Quantifies adherence to time/space complexity requirements, normalized against reference solutions.
- **Mean Average Precision (mAP)**: A ranking metric that averages precision across recall levels, often used in information retrieval settings [16].
- **Memory Usage**: Measures the memory footprint of generated code during execution [65].
- **Readability**: Assesses how easy the generated code is to read and understand, often based on heuristics or human ratings [216].
- **Precision@k, Recall@k and F1-score**: Widely used to evaluate classification performance. [93, 104, 203].
- **Reuse Correctness**: Evaluates whether reused components or functions are applied correctly in the generated solution [104].
- **Runtime**: The total execution time of the generated solution under test [65, 129].
- **Speedup@k**: Measures performance improvement (e.g., in runtime or accuracy) among the top- k generated solutions relative to a baseline [130].

In addition, a range of similarity-based metrics have been employed to compare generated code with the corresponding canonical solutions. These include **BLEU** [29, 48], **METEOR**, **ROUGE**, **CodeBLEU** [178] and **ChrF** [16]. These metrics aim to quantify the lexical or semantic similarity between model-generated outputs and reference implementations.

For tasks such as *code translation*, *code editing*, *code summarization*, *commit message generation*, and *software engineering question answering*, a combination of natural language processing (NLP) metrics and executable validation metrics is used. These metrics evaluate either textual similarity to reference outputs or the functional correctness and executability of generated code. Metrics include:

- **BLEU**: A token-level n -gram precision metric used to assess textual similarity [4, 62, 76, 132, 190, 191].
- **CodeBLEU**: An extension of BLEU that incorporates syntactic and semantic code structure for more accurate evaluation of generated programs [4, 76, 174, 191].

- **ROUGE-L**: A recall-oriented metric based on the longest common subsequence, used for evaluating NLP tasks [62, 132, 190].
- **BERTScore**: A semantic similarity metric that uses contextual embeddings from pre-trained language models to compare generated and reference texts [16, 190].
- **Exact Match**: Measures whether the generated output exactly matches the reference string, commonly used in classification-style or answer-extraction tasks [191].
- **Data Flow Match**: Evaluates the consistency of data flow between the generated and reference code, emphasizing semantic correctness [4].
- **Build@k**: Indicates whether at least one of the top- k generated outputs can be successfully built (i.e., compiled and linked without errors) [174].
- **Debugging Success Rate (DSR)@k**: Measures the percentage of samples among all evaluated instances for which at least one of the top- k generated patches executes successfully and produces the expected results [134, 191].

It is worth noting that several execution-based evaluation metrics are also employed in these tasks. These include **Pass@k** [190], **Pass Rate** [134, 174], **Edit Similarity** [62], and the **Average Test Case Pass Ratio** [174].

Benchmarks focusing on test case generation and test case repair employ a diverse set of metrics to evaluate code coverage, correctness, applicability, and repair of faulty or incomplete test cases. The following metrics are used:

- **Branch Coverage**: Assesses the percentage of decision branches exercised by the generated test cases [172, 212].
- **Change Coverage**: Measures the extent to which generated tests cover modified code regions [126].
- **Compilation Rate**: Proportion of generated test cases that compile successfully [212].
- **Cov@k**: Coverage at top- k , quantifying how many unique coverage elements (e.g., lines or branches) are triggered within the top- k generated test cases [71, 172, 212].
- **Fail-to-Pass Rate**: Percentage of failing test cases that are repaired into passing ones [7].
- **Line Coverage**: Measures the proportion of code lines executed by the generated test suite [172, 212].
- **Mutation Score**: Evaluates the ability of test cases to detect faults by measuring how many mutants are killed [172].
- **Pass@k**: Proportion of test generation attempts in which at least one of the top- k test cases is functionally correct [71].
- **Patch Applicability**: Checks whether generated test cases are compatible with and applicable to the target patched code [126].
- **TDD Score**: Evaluates the quality of generated tests over a set of instances by computing the arithmetic mean of per-instance scores, scaled between 0 and 100; higher scores indicate better alignment with expected testing behavior [7].
- **Win Rate**: Represents the percentage of instances where a model's output is judged as more closely aligned with the ground truth compared to a competing model. This metric is often derived from GPT-based evaluators shown to approximate human judgment effectively and efficiently [15].
- **DiffBLEU**: Measures BLEU score over diff-like edits between faulty and repaired test cases [112].
- **Syntax Pass Rate**: Percentage of repaired test cases that are syntactically valid and compile without errors [112].

In addition to the above metrics, metrics such as accuracy [112, 172, 212], success rate [126], and CodeBLEU [112] were employed too.

Issue Resolution Tasks. For benchmarks targeting issue resolution and bug-fixing tasks, several metrics are employed to quantify success and resource efficiency:

- **% Resolved:** Measures the proportion of issues correctly resolved by the model [8, 77, 193, 206].
- **Average Cost:** Reflects the average cost and financial resources required to resolve an issue [8, 193, 206].

Several benchmarks designed for automated program repair evaluate the correctness, plausibility, and efficiency of generated patches using the following metrics:

- **AST Match@1:** Determines whether the top-1 generated patch structurally matches the abstract syntax tree (AST) of the ground-truth fix [158].
- **Number of Correct Fixes:** Counts the total number of correct patches generated by the model [75].
- **Plausible@1:** Indicates the percentage of instances where the top-1 patch compiles and passes at least one test case, even if not fully correct [158].
- **Count@k:** Captures the number of problems for which at least one of the k generated patches is considered plausible, i.e., functionally valid [207].

Beyond the previously discussed metrics, several general-purpose evaluation measures have also been employed in the context of program repair tasks. These include **Accuracy** [114], **Pass Rate** [163], **Pass@1** [114], and **Average Cost** [158].

Benchmarks focusing on fixing bugs in code, such as Bug Repair, Detection, and Fault Localization, evaluate the correctness and effectiveness of generated patches using the following metrics:

- **Top-5 Accuracy:** Measures whether the correct fix appears among the top 5 generated solutions [192].
- **Avg-5:** Reports the average score or accuracy over the top 5 generated patches, reflecting overall repair quality [192].
- **RPSR (Relative Patch Size Ratio):** Represents patch precision by quantifying the edit efficiency of a generated patch relative to the ground truth patch, using Tree Edit Distance between the original buggy code and each patch. [192].

Alongside these, **Pass@k** [45] and **Success Rate** [11] have also been used to evaluate the effectiveness of bug repair systems. For code reasoning tasks, the evaluation has mainly relied on **Accuracy** and the **Incremental Consistency (IC) Score** [26], which quantifies the model's ability to maintain logical consistency in a sequence of tasks with increasing levels of difficulty. Additionally, **Pass@k** has been employed in the *CruxEval* benchmark to assess functional correctness on multiple attempts [53].

Finally, for the formal verification task, **Success Rate** has been used as the primary metric, measuring the proportion of verification problems successfully solved by the model [117], and MobileBench used **Pass Rate** and **CheckPoint** which validates software workflows by verifying package names, key UI phrases and API calls against predefined sequences [38].

4.3.2 Are there any standardized evaluation methodologies associated with these datasets? Standardization of the evaluation methodology is critical to ensure the reproducibility, comparability, and fairness of LLM benchmarking efforts. To assess whether such standardization exists, we examined the output format and evaluation processes used in the surveyed 156 benchmark datasets. Our analysis revealed six different categories of evaluation methodology, reflecting the diversity of tasks and the varied levels of methodological rigor.

Similarity-based Evaluation: These evaluation frameworks rely on NLP techniques to assess the similarity between the model-generated output and the reference solution. Such approaches, employed in 17 benchmark studies, typically measure token-level or structure-level similarity. Commonly used metrics include **BLEU**, **CodeBLEU**, **DiffBLEU**, **Edit Distance**, and **Exact Match**, as well as semantic similarity metrics such as

Table 14. Detailed distribution of benchmark evaluation metrics

No	Source	# Studies	References
1	% Resolved	6	[8, 77, 124, 193, 205, 206]
2	Acceptance Rate	1	[154]
3	Accuracy	33	[4, 5, 12, 26, 27, 35, 43, 73, 76, 82, 91, 95, 104, 112, 114, 116, 118, 120, 121, 128, 133, 152, 154, 159, 162, 166, 179, 190, 196, 198, 207, 215–217]
4	Api Misuse Rate	1	[220]
5	Ast Match@1	1	[158]
6	Avg Generated Solution Complexity	1	[139]
7	Avg Generated Reference Complexity	1	[139]
7	Avg Cost	5	[8, 129, 158, 193, 206]
8	Avg Speedup	1	[159]
9	Avg Step	2	[38, 68]
10	Avg-5	1	[192]
11	Bertscore	3	[16, 146, 190]
12	Beyond(Efficiency Score)	1	[41]
13	Bleu	13	[16, 48, 62, 76, 120, 132, 152, 166, 178, 190, 191, 200, 217]
14	Build Repair Rate	1	[16]
15	Build@k	1	[174]
16	Code Coverage	1	[126]
17	Checkpoint	1	[38]
18	Chrf	2	[16, 178]
19	Code Lines	1	[186]
20	Code Tokens	1	[186]
21	Codebleu	9	[48, 76, 112, 116, 120, 152, 178, 191, 217]
22	Communication Rate	1	[181]
23	Compilation Rate	3	[113, 212, 220]
24	Compilation@k	2	[20, 142, 207]
25	Completion Rate	1	[68]
26	Completion@k	1	[20]
27	Correctness	6	[57, 113, 145, 172, 172, 212]
28	Cov@k	6	[71, 113, 145, 172, 207, 212]
29	Data Wrangling Score	1	[68]
30	Debugging Success Rate@k	3	[134, 190, 191]
31	DiffBLEU	2	[112, 159]
32	Differential Performance Score	1	[111]
33	Eda Score	1	[68]
34	Edit Distance	1	[179]
35	Edit Similarity	6	[40, 48, 62, 116, 120, 194]
36	Efficiency Score	3	[65, 91, 216]
37	Efficiency@k	2	[130, 141]
38	Exact Match	7	[16, 40, 116, 120, 132, 152, 191]
39	Executability Rate	1	[129]
40	F1 Score	20	[5, 12, 16, 26, 40, 48, 56, 82, 84, 101, 104, 107, 118, 120, 133, 161, 166, 190, 198, 215]
41	Failtopass	1	[7]
42	Good Question Rate	1	[181]
43	Incremental Consistency Score	1	[26]
44	LLM as a Judge	1	[15]
45	Machine Learning Score	1	[68]
46	Maintainability Index	1	[216]
47	Map	1	[16, 120]
48	Medal Rate(like Kaggle)	1	[24]
49	Memory Usage	1	[65]
50	Meteor	3	[166, 178, 190]
51	MRR	4	[58, 84, 120, 222]
51	Mutation Score	1	[172]
52	Number Of Correct Fixes	1	[75]
53	Number Of Errors	1	[129]
54	Pass Rate	9	[33, 38, 41, 99, 121, 131, 139, 163, 180, 181]
55	Pass@1	13	[23, 55, 59, 72, 87, 114, 115, 134, 138, 139, 179, 185, 194]
56	Pass@k	51	[13, 14, 20, 22, 24, 25, 29, 31, 34, 42, 45, 53, 58, 60, 63, 65, 71, 82, 86, 90, 93, 94, 97, 98, 100, 102, 110, 111, 122, 130, 131, 140, 142, 156, 167, 171, 175, 176, 178, 187, 188, 190, 200, 202–204, 207, 217, 220, 224, 225]
58	Patch Applicability	1	[126]
59	Plausible@1	1	[158]
60	Precision	15	[5, 12, 16, 43, 56, 99, 101, 104, 107, 118, 121, 133, 166, 190, 198]
61	Readability	1	[216]
62	Recall	17	[5, 12, 16, 43, 48, 56, 58, 99, 101, 104, 107, 118, 121, 133, 166, 190, 198]
63	Recall@k	2	[93, 222]
64	Rouge	4	[16, 166, 178, 190]
65	Rouge-1	3	[62, 95, 132]
66	RFSR	1	[192]
67	Runtime	3	[65, 129, 206]
68	Speedup@k	1	[130]
69	Success Rate	7	[4, 11, 117, 126, 160, 174, 186]
70	TDDScore	1	[7]
71	Token Usage	1	[129]
72	Top-5	1	[192]
73	Win Rate	1	[15]
-	Unique Total	142	-

BERTScore. These metrics are particularly well-suited for tasks such as code translation, summarization, and synthesis, where functional executability is not strictly required.

Execution-Based Evaluation. The most widely adopted evaluation methodology across code generation benchmarks is execution-based evaluation, observed in more than 90 benchmarks. This approach assesses the functional correctness of the generated code by executing it against a predefined set of test cases. It is considered the most reliable method to evaluate the practical utility of code generation models [14, 29, 60]. Execution-based evaluation involves several strategies. Unit test pass rate measures the proportion of generated solutions that pass all predefined test cases, using metrics such as **Pass@k**, **Pass@1**, and **Success Rate** [29]; compilation rate evaluates whether the generated code is syntactically and semantically valid by attempting to compile it, with metrics such as **Compilation@k** quantify the percentage of compilable outputs within the top-*k* candidates [207]; performance and efficiency evaluation assesses the runtime characteristics of generated code, including computational efficiency and memory consumption, using metrics such as **Efficiency@k**, **Runtime**, and **Memory Usage** [65, 216]; and task resolution rate measures the model’s ability to solve real-world tasks such as bug fixing or issue resolution, typically quantified through metrics such as **Pass Rate** or the **percentage of resolved issues** [77, 134]. Execution-based evaluation provides strong guarantees of functional correctness, particularly in settings where code must be executable and produce meaningful outcomes. It is especially appropriate for synthesis, repair, and debugging tasks, where correctness cannot be inferred solely from textual similarity.

Human-Feedback Evaluation: In this evaluation strategy, human annotators are involved in manually assessing the quality of outputs generated by LLMs. These benchmarks typically define a structured annotation pipeline that includes a rubric for scoring aspects such as correctness, readability, relevance, and overall quality. In addition, human evaluations may include statistical analyses to ensure the reliability and validity of the assessment results [129]. Peer review and majority voting are often employed to ensure annotation consistency. Human-feedback-based evaluation provides high-fidelity judgments, especially for tasks where automated metrics may fall short, such as code summarization, documentation generation, or question answering [154].

LLM-Based Evaluation: This method uses large language models themselves as evaluators, commonly referred to as the *LLM-as-a-Judge* paradigm. In these benchmarks, the LLM is provided with sufficient context, instructions, and an evaluation rubric, enabling it to assess the quality of model-generated outputs. This approach offers a scalable and low-cost alternative to human annotation and has been shown to correlate well with human judgment in various tasks [154, 215]. It is particularly useful in scenarios where interpretability and rapid feedback are required.

Multiple-Choice and Question Answering Evaluation: This category includes benchmarks structured as multiple choice, fill-in-the-blank, or classification-based tasks. Models are evaluated based on standard classification metrics such as **Accuracy**, **F1 Score**, or **Exact Match**. This method is widely used in *software engineering question answering* and *program understanding* tasks [62, 128].

Ad Hoc Evaluation: This group consists of evaluation methodologies that are tightly coupled with the unique structure or objectives of specific benchmarks. While they may share some metrics with other categories (e.g., correctness or consistency), their design involves formal methods, specification matching, or logic-based reasoning pipelines. Due to the task-specific nature of these evaluations, they are categorized separately to highlight their distinctive evaluation requirements [26, 117].

4.3.3 Critical Assessment and Recommended Evaluation Suite. While our analysis highlights a wide range of metrics, the current landscape is heavily skewed toward functional correctness. The dominance of *Pass@k*, appearing in 51 studies, indicates that there is a community-wide optimization for “test-passing” capabilities. However, relying solely on *Pass@k* presents significant risks:

- **Overfitting to Functional Correctness:** High *Pass@k* scores can often mask poor coding practices. Models might generate functionally correct code that is structurally complex, hard to maintain, or insecure. This

narrow focus on functional correctness allows models to achieve high scores without demonstrating adherence to best practices.

- **Blindness to Code Quality:** *Pass@k* suffers from a significant disconnect regarding non-functional requirements, where metrics for efficiency, readability, and safety are underrepresented compared to correctness metrics. These aspects are largely absent from general-purpose coding benchmarks, appearing primarily in niche datasets.

To address these gaps and promote standardized evaluation, we propose a minimal recommended suite for future benchmarks, provided in Table 15. Based on the metric types identified in our survey, we categorize five essential dimensions necessary for the comprehensive evaluation of various code intelligence tasks:

Table 15. Recommended Suite for Evaluating Code Intelligence Tasks

Dimension	Definition	Recommended Metrics
Correctness	The functional adherence of the code to the problem description.	<i>Pass@k</i>
Robustness	The ability of the code to handle edge cases, changes, or stress.	Mutation Score, Fail-to-Pass Rate.
Efficiency	The computational and resource performance of the generated solution.	Normalized Efficiency Index, Runtime/Memory Usage.
Readability	The ease with which human developers can understand and maintain the code.	Maintainability Index, Cognitive Complexity.
Safety	The detection of security flaws and vulnerabilities.	Vulnerability Detection Rate (e.i. Precision, Recall, F1-score)

Answer to RQ3:

- (1) Although the choice of evaluation metric typically depends on the specific code intelligence task being addressed, *Pass@k* and accuracy emerge as the most widely adopted metrics across studies.
- (2) Multiple evaluation pipelines are used depending on the nature of the task; among these, execution-based evaluation is the most prevalent, evaluating the functional correctness or other metrics of generated code by executing it against predefined test cases.

4.4 RQ4: Challenges, Gaps, and Limitations

4.4.1 Methodology for Identifying Shortcomings. To identify the limitations of existing benchmarks systematically, we adopt a *community-driven* methodology that synthesizes challenges reported by researchers who have empirically evaluated these datasets. Rather than relying on subjective assessment, our analysis is grounded in peer-reviewed evidence, ensuring that the identified shortcomings reflect practical issues encountered by the research community when applying these benchmarks to evaluate LLMs.

Dataset Selection. Our dataset selection strategy was designed to balance representativeness, impact, and evidence availability. We initially aimed to select the top three most cited benchmarks within each phase of the Software Development Life Cycle to ensure broad coverage of diverse code intelligence tasks. However, an exploratory analysis of the literature revealed a clear imbalance across SDLC phases, that benchmarks associated with the *implementation* phase, and in particular the *code generation* task, are substantially more prevalent and more actively studied than benchmarks in other phases. To avoid including benchmarks with insufficient empirical discussion of their limitations, we refined our selection criteria to focus on datasets that (i) demonstrate

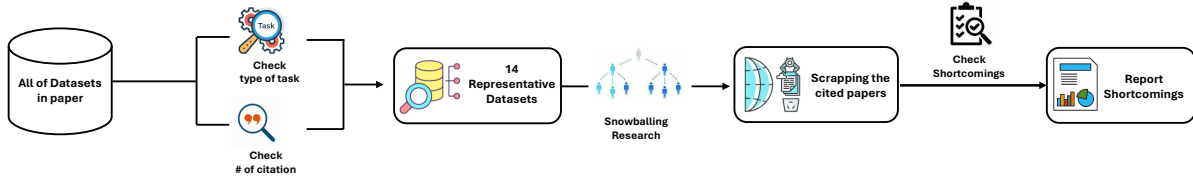


Fig. 7. Shortcoming identification workflow

strong adoption within the community, (ii) have high citation counts, and (iii) are supported by a sufficiently rich body of literature explicitly discussing their weaknesses. In addition to widely established benchmarks, we deliberately included high-impact and niche benchmarks within the code generation domain to capture emerging evaluation settings and task formulations that are shaping current research. This refinement process resulted in a final set of 14 representative benchmarks, providing a comprehensive and evidence-rich view of the current benchmarking landscape.

Data Collection and Analysis. To extract benchmark shortcomings reported by the community, we employed a systematic *snowballing* strategy consisting of the following steps:

- (1) **Citation Crawling.** Using the same automated crawling infrastructure described in Section 3, we retrieved all academic publications that cite the 14 selected benchmarks.
- (2) **LLM-Assisted Filtering.** Given the large volume of citing papers, we leveraged LLMs to perform an initial screening step, identifying publications that explicitly mention limitations or shortcomings of the referenced benchmarks.
- (3) **Manual Extraction and Validation.** To ensure reliability and mitigate potential hallucinations introduced during automated filtering, the final extraction was performed manually. One author reviewed the flagged papers to extract and categorize the reported shortcomings, while a second author independently verified the extracted information.

Figure 7 illustrates the workflow of our methodology.

4.4.2 What limitations do these datasets have in terms of coverage, bias, and applicability? Based on our analysis of the reviewed papers, we have classified the shortcomings of the representative datasets into the following categories. A detailed mapping of these limitations, along with the corresponding papers that identify them, is provided in Table 16.

- **Limited Task Complexity and Real-World Relevance:** The most frequent challenge in many existing LLM benchmark datasets is their insufficient task complexity and limited alignment with real-world code intelligence scenarios. Numerous benchmarks are primarily designed to assess LLMs on simplified algorithmic or competitive programming tasks, which, while useful for controlled evaluation, fail to reflect the multifaceted challenges encountered in practical software development [129, 188]. Similarly, datasets focused on data science often emphasize basic tasks, overlooking more complex and realistic components such as data visualization, data understanding [9, 54].

Furthermore, many benchmarks are restricted to single-function or single-file contexts, neglecting the intricate dependencies, module interactions, and architectural considerations inherent in large-scale, multi-file software projects [37]. In the domain of test generation, benchmarks frequently limit evaluation to the generation of isolated test cases rather than comprehensive and structured test suites [71]. Additionally, a portion of existing datasets are centered exclusively around the Python ecosystem, limiting the generalizability of evaluation outcomes [206]. This single language focus overlooks the diversity of

programming languages and development environments encountered in real-world code intelligence tasks, thereby constraining the external validity of LLM performance assessments.

- **Data Quality:** The second most commonly observed limitation in LLM benchmark datasets is the lack of data quality. Benchmarks rely on LLMs themselves to generate dataset content, often without sufficient human supervision or validation, which can result in low-quality or unreliable data [169, 208]. In other cases, the selection of tasks, whether from online repositories or through manual curation, lacks a balanced distribution across different complexity levels, with a noticeable bias toward short and simpler questions [3, 39]. This skew limits the benchmarks' ability to meaningfully differentiate LLM capabilities across varying levels of difficulty.

Moreover, many benchmarks suffer from poorly constructed task descriptions. These descriptions are often too brief, lack comprehensiveness, or contain ambiguous or imprecise definitions of the problem [110, 157]. Such deficiencies in task specification can confuse LLMs during inference and adversely affect performance, not due to model limitations but due to inadequate problem formulation. This undermines the reliability of the benchmark as an accurate measure of LLM performance in realistic settings.

- **Inadequate Test Coverage:** Another notable limitation in existing LLM benchmark datasets lies in the inadequacy of test cases used to evaluate model performance. Many datasets suffer from insufficient test coverage, failing to assess the full range of correctness, robustness, and efficiency of the generated solutions. Common issues include the presence of incorrect or poorly designed test cases that fail to cover all edge cases and scenarios for comprehensive testing [79, 108, 110]. Another notable issue is the presence of incorrect or flawed canonical solutions within some benchmarks, which can compromise the reliability of evaluation results. For instance, such errors have been observed in the HumanEval dataset, where inaccuracies in the reference solutions have led to misleading assessments of model performance [110]. In some datasets, there is an absence of stress tests that could evaluate aspects such as runtime performance, leading to further quality degradation. These limitations hinder the benchmarks' ability to effectively and thoroughly validate generated code [79, 141]. As a result, code may contain minor problems such as poor syntax or severe flaws, and semantic errors like uncaught bugs in edge cases. These unnoticed issues can undermine the reliability and diagnostic power of the benchmark.
- **Evaluation Limitations:** A further limitation observed in several LLM benchmark datasets is the lack of a comprehensive and well-structured evaluation framework capable of assessing the full range of generated outputs. Many benchmarks fail to adopt a holistic evaluation approach that captures multiple dimensions of code quality, such as correctness, efficiency, readability, and robustness. This shortcoming undermines the ability of these datasets to fulfill their intended purpose as reliable tools for evaluating LLM performance. Specific issues include the absence of a systematic evaluation process [210], reliance on limited subsets of the dataset for evaluation [7], execution environment misalignment, and differences in hardware might affect evaluation metrics about performance [143], and the inability of evaluation pipelines to simulate the complexity and interdependencies of real-world software development. These deficiencies reduce the applicability of such benchmarks for realistic and practical LLM evaluation.
- **Data Leakage Risks:** Several benchmarks are vulnerable to data leakage, which can significantly compromise the validity of LLM performance evaluations. The use of publicly available repositories in dataset construction increases the risk of overlap with training data, particularly given that many LLMs are trained on large-scale public code corpora [8, 123, 157]. Additionally, the inclusion of data created before the training cut-off dates of LLMs further heightens this risk. These forms of leakage can lead to inflated performance estimates and misrepresent the true capabilities of the models being evaluated.

4.4.3 How do these limitations impact the evaluation and interpretation of LLM performance?

Table 16. Limitations of LLM benchmark datasets in code intelligence

Category	Dataset	# Studies	References
Limited Task Complexity and Real-World Relevance	TestEval	10	[71]
	ARCADE		[9, 54, 92]
	HumanEval		[129, 188, 213, 217]
	MBPP		[129]
	CruxEval		[108]
	DS-1000		[33, 50, 79]
	DebugBench		[196, 197]
	SWTBench		[10]
	SWEBench		[206]
Data Quality	CrossCodeEval	9	[37]
	API-Bank		[169, 208]
	CruxEval		[106]
	LiveCodeBench		[39]
	CodeContests		[3]
	SWT-Bench		[127]
	ARCADE		[79]
	HumanEval		[110]
	MBPP		[157, 165]
Evaluation Limitations	DS-1000	8	[51]
	SWEBench Multimodal		[210]
	LiveCodeBench		[143]
	CruxEval		[121]
	ApiBank		[2, 119]
	MBPP		[83]
	HumanEval		[110]
	CodeContests		[97]
Inadequate Test Coverage	SWTBench	7	[7]
	LiveCodeBench		[195]
	HumanEval		[110, 213]
	MBPP		[108]
	CodeContests		[147, 177]
	DS-1000		[79]
	SWEBench		[8]
Data Leakage Risks	TestEval	4	[141]
	MBPP		[122]
	HumanEval		[157]
	SWEBench		[8]
Unique Total	LiveCodeBench	14	[123, 182]
	-		-

- Limited Task Complexity and Real-World Relevance** It is evident that overly simplistic tasks confine the ability of LLMs to show their true potential. In such settings, the reasoning capabilities of LLMs are underutilized, as tasks often do not require complex logic, design patterns, or architectural understanding. This simplicity can lead to artificially inflated performance, thereby misrepresenting the true capabilities of the models in real-world code intelligence contexts [213].
 Moreover, focusing on a single programming language, most commonly Python [217], and single-file tasks limits the assessment of the generalizability of LLMs across diverse codebases. Real-world software development typically involves multi-language ecosystems, cross-file dependencies, and the need to navigate and modify large, structured codebases. Benchmarks that do not reflect these conditions fall short in assessing the practical utility of an LLM in professional production-level environments [40].
- Data Quality** Bias toward simpler and shorter tasks, along with insufficient dataset quality, will fail to reflect how LLMs would perform in practical scenarios and can result in artificially inflated performance metrics for LLMs. In addition, ambiguity or imprecision in problem descriptions may lead LLMs to produce incorrect solutions, as they may misinterpret the task requirements. This affects the validity of

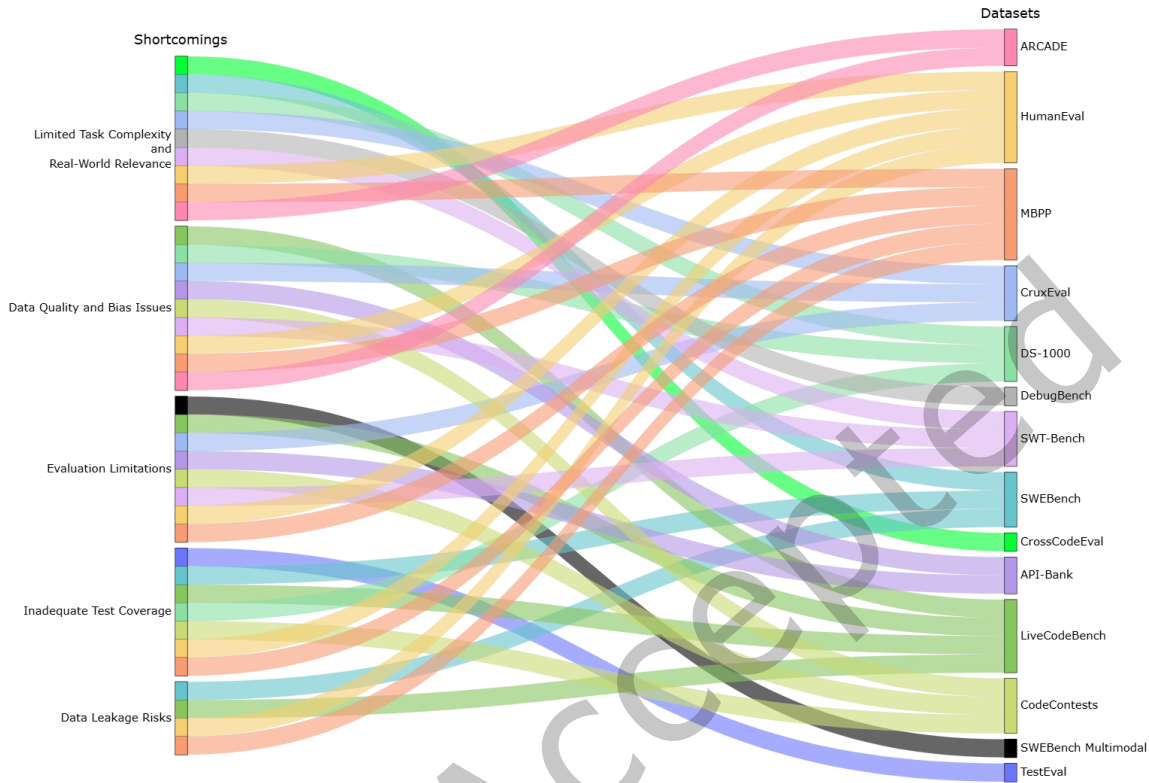


Fig. 8. Limitations of LLM benchmark datasets in code intelligence

the evaluation and can contribute to hallucination or overconfidence in incorrect results, reinforcing flawed reasoning patterns within models [157].

- **Inadequate Test Coverage** The lack of high-quality test cases represents another critical limitation that can adversely affect the evaluation of LLM performance. Inadequate test coverage can result in undetected bugs within generated code, leading to the acceptance of flawed or buggy solutions that could have serious consequences in real-world scenarios [177, 195]. Furthermore, the absence of stress testing hinders the ability to assess the robustness of LLMs under realistic conditions, where attributes such as scalability, performance under load, and fault tolerance are essential for practical deployment [79].
- **Evaluation Limitations** The absence of a well-defined and comprehensive evaluation framework for benchmark datasets may lead researchers to adopt ad hoc evaluation methods, therefore restricting the comparability of results and limiting the ability to draw consistent and reliable conclusions about the performance of different LLMs on the same dataset [210].
- **Data Leakage Risks** Data leakage can result in unintended memorization by the model, leading to artificially inflated performance metrics that do not accurately reflect the true capabilities of the model

[221]. This risk is especially pronounced for widely circulated benchmarks like HumanEval, as their content may have been accidentally incorporated into LLM training data, resulting in artificially high scores that do not reflect genuine real-world generalization [122].

Figure 8 provides a visual representation of the identified shortcomings and the representative datasets in which they were observed.

Answer to RQ4:

- (1) The shortcomings of current benchmarking datasets can be categorized into five major areas: limited complexity and real-world relevance, biases in data selection that compromise quality, structural limitations of LLMs that constrain their suitability for evaluation, insufficiently thorough or high-quality test cases, and risks of data leakage.
- (2) These limitations can lead to significant challenges in evaluating LLMs, including producing inflated or unrealistic results that overstate model capabilities, failing to capture performance in low-resource programming languages, and encouraging hallucination or overconfidence due to imprecise task descriptions.

4.5 RQ5: Future Directions

Based on our systematic analysis of existing LLM benchmarks in code intelligence, we identify five critical directions for advancing evaluation frameworks to meet the field’s evolving demands. Each direction addresses the gaps identified in current methodologies while aligning with emerging requirements for specialized, scalable, and context-aware assessments.

4.5.1 Multi-Modal Benchmark Development. Although early efforts like SWE-Bench Multi-Modal [193] have begun exploring multi-modal evaluation, significant gaps remain in assessing LLMs’ ability to process heterogeneous code intelligence artifacts. Future benchmarks should integrate code with complementary modalities, including UI/UX designs (e.g., Figma mockups), system architecture diagrams, and runtime execution traces, particularly for front-end development tasks where visual elements and interactive behaviors co-evolve with code [184, 209]. New datasets could combine version control histories with design system changes to enable evaluation of cross-modal reasoning capabilities. A pressing need exists for standardized metrics that quantify alignment between code modifications and corresponding interface updates, as current benchmarks focus primarily on textual quality assurance without addressing visual artifacts. This direction is critical for applications requiring coordination between design systems and implementation, such as responsive web development and accessibility compliance.

Practical Challenges. The key challenge in multi-modal benchmarking arises from the high cost of collecting and aligning heterogeneous data (e.g., synchronizing UI designs with codebase versions). Manual curation is already resource-intensive; adding visual modalities increases this burden and introduces subjectivity in evaluation (e.g., judging “visual correctness”).

Mitigation Strategies. Recent advances in LLMs enable scalable alternatives to manual curation. Synthetic data generation can produce code from structured UI descriptions, while automated rendering and validation tools support efficient assessment of visual outputs. In addition, crowdsourcing pipelines leveraging front-end developer communities can help distribute data collection and annotation costs, improving scalability while reducing reliance on expert-driven curation.

4.5.2 Domain-Specific Benchmark Specialization. Current benchmarks exhibit excessive generality, limiting their utility for specialized code intelligence subdomains. We advocate for benchmarks tailored to back-end development (e.g., API design and distributed system patterns), DevOps (CI/CD pipeline synthesis and

infrastructure-as-code validation), and front-end tasks (design-to-code translation and cross-browser compatibility resolution). For instance, SWE-Bench [77] demonstrates the value of GitHub issue-based evaluations but lacks specialization for domain-specific workflows. Datasets like DS-1000 [87] for data science libraries highlight the benefits of targeted evaluations, yet coverage remains sparse for critical domains like deep learning frameworks.

Practical Challenges. Specialization requires deep domain expertise (e.g., identifying valid DevOps pipeline configurations), which is scarce and expensive to annotate. Furthermore, domain-specific tools evolve rapidly; a benchmark for a specific CI/CD tool may become obsolete within months, leading to high maintenance overhead.

Mitigation Strategies. To address expertise scarcity, benchmarks should utilize mining of official documentation and migration guides to automatically generate ground-truth pairs. To handle tool evolution, the community should adopt dynamic versioning, where the benchmark metadata explicitly tags the tool version, allowing for version-specific leaderboards rather than a single static score.

4.5.3 Large-Scale and Real-World Benchmarks. Existing benchmarks like SWE-Bench [77] capture only partial aspects of industrial-scale software maintenance. Next-generation datasets must incorporate multi-repository dependencies mirroring microservice architectures, legacy system migration scenarios with deprecated dependencies, and cross-version API compatibility challenges. Furthermore, there is an urgent need for benchmarks targeting low-resource programming languages like Go and Rust, which power critical infrastructure but lack adequate evaluation frameworks. The MultiPL-T [21] approach, which generates validated training data through code translation and test synthesis, offers a promising pathway for creating benchmarks in resource-constrained environments; however, code translation fails to capture language-specific paradigms, runtime behaviors, and idiomatic coding practices unique to emerging languages like Go and Rust.

Practical Challenges. Simulating realistic environments is computationally expensive and prone to “flakiness” (non-deterministic execution errors), as seen in the complexity of existing benchmarks like SWE-bench [77]. The infrastructure cost for executing multi-container environments for every evaluation run is challenging for many academic labs.

Mitigation Strategies. Benchmarks should be distributed as pre-built, lightweight Docker containers with fixed dependencies to ensure reproducible evaluation. To further manage computational cost, tiered evaluation settings can be provided, including a *Lite* configuration for rapid iteration with mocked dependencies and a *Full* configuration for final validation using complete environment execution.

4.5.4 Benchmarks for Assessing LLM Reasoning Abilities. As LLMs advance in their capacity for complex reasoning, it becomes increasingly important to develop benchmark datasets that accurately assess their problem-solving abilities across diverse contexts. Effective reasoning benchmarks must go beyond simple code generation or single-file edits, incorporating multi-step reasoning, cross-context comprehension, and agentic behavior to evaluate the true capabilities of LLMs.

For instance, benchmarks like SWE-Bench [77], which includes multi-file edits, and RepoBench [116], designed for cross-context code completion, represent initial steps toward this goal. However, these benchmarks lack standardized protocols for assessing agentic reasoning, such as the ability to plan, navigate dependencies, and make strategic decisions when encountering novel or ambiguous scenarios. To comprehensively evaluate reasoning abilities, future benchmarks should incorporate:

- **Multi-step Reasoning:** Tasks that require models to solve problems incrementally, integrating intermediate steps to reach a final solution. This approach is essential for testing models on real-world challenges where solutions are rarely straightforward.

- **Cross-context Comprehension:** Scenarios where models must understand and reason across multiple files, functions, or data sources, reflecting the interconnected nature of complex software systems.
- **Strategic Decision-Making:** Evaluating the capacity of the model to plan and adapt based on the evolving context of the task, including the ability to prioritize, manage uncertainty, and handle conflicting requirements.
- **Agentic Behavior:** Tests that assess the ability of LLMs to operate independently, make context-aware decisions, and adapt to dynamic environments without direct supervision, reflecting their potential role as autonomous agents in code intelligence tasks.

Establishing such benchmarks requires the development of standardized evaluation protocols, robust metrics, and carefully curated datasets that accurately reflect the challenges faced by modern software engineers. These frameworks should also address the limitations of existing benchmarks, including the need for real-world task diversity, realistic test cases, and rigorous assessments of the depth of reasoning.

***Practical Challenges.** Defining and evaluating reasoning remains inherently subjective, and manual annotation of intermediate reasoning steps (e.g., chain-of-thought) is both time-consuming and difficult to scale. Moreover, models may learn to imitate surface-level reasoning patterns rather than demonstrating genuine logical understanding.*

***Mitigation Strategies.** To address these challenges, evaluation should move beyond static text comparison toward interactive or execution-based verification, where models must demonstrate reasoning by successfully engaging with compilers or debuggers (e.g., predicting the output of an execution trace). For data construction, knowledge distillation from stronger reasoning models can further reduce reliance on costly manual annotation.*

4.5.5 Cutoff-Aware Benchmark Collection. To ensure a reliable and fair evaluation of LLMs, benchmark datasets must be curated with strict awareness of the training data cutoff date. Including data that predates this cutoff risks introducing data contamination, wherein the model has already seen the evaluation content during training, leading to inflated performance metrics.

- **Temporal Data Filtering:** All benchmark samples, such as code snippets, problem descriptions, and documentation, should be verified to postdate the training cutoff of LLMs. This prevents inadvertent overlap between training and evaluation data. Previous studies have demonstrated that LLMs can memorize and reproduce benchmark content if it appears in the training data [148].
- **Source Validation:** When using data from popular online platforms such as GitHub, LeetCode, extra caution is warranted. These platforms are common sources for pre-training corpora and may inadvertently contain benchmark-relevant material. Proper filtering or high-quality synthetic data generation is required to reduce the risk of contamination [36].
- **Dynamic Benchmarking:** Benchmarks should be designed to evolve and remain relevant for newly released LLMs. This requires infrastructure that can collect fresh, high-quality data published after model-specific training cutoffs. Such dynamic benchmarks help maintain the validity of the evaluations as the LLMs and training datasets grow rapidly.

By collecting benchmark data after the training cutoff date of models and validating the sources used, researchers can help ensure that performance evaluations are not affected by prior model exposure.

***Practical Challenges.** The cutoff is a moving target as new models are released weekly. Maintaining a benchmark that is guaranteed to be unseen requires constant and continuous data collection, which is a complex process.*

***Mitigation Strategies.** This challenge can be mitigated through automated and continuous benchmark updates. For example, a crawler can periodically collect trending repositories, filter them by commit time, and apply a*

standard test-generation pipeline. Automating this process reduces manual effort and helps maintain a rolling set of recently created evaluation data.

4.5.6 Test Design Integrity and Robust Evaluation Protocols. In addition to temporal considerations, benchmarks must be constructed with meticulous supervision to ensure a comprehensive evaluation of LLM reasoning and generalization and to avoid data leakage.

- **Test Case Integrity:** Evaluation tasks should be carefully designed to avoid leaking answer-related information. Inputs should not inadvertently reveal ground-truth solutions, and metadata should be controlled to ensure a fair assessment.
- **Comprehensive Coverage:** Benchmarks should include a diverse set of examples that cover easy, typical, and even edge-case scenarios. Covering these categories helps evaluate how models generalize beyond common patterns and whether they can handle complex inputs.
- **Continuous Monitoring:** Dataset integrity is not a one-time concern. Ongoing audits and validation protocols should be instituted to monitor for emerging data leaks or inadvertent contamination in future benchmark updates or extensions.

These protocols strengthen the robustness and interpretability of benchmark results, ensuring that LLM performance metrics genuinely reflect their problem-solving and reasoning capabilities under uncontaminated conditions in the real world.

Practical Challenges. Developing high-coverage test suites at scale is costly and time-consuming, and many existing datasets lack sufficient quality control.

Mitigation Strategies. This issue can be mitigated by using automated checks to assess test effectiveness and by combining LLM-generated tests with selective human review. For instance, candidate tests can be LLM-generated and evaluated against reference implementations. At the same time, human effort is focused only on ambiguous or failing cases, reducing manual workload while maintaining test quality.

4.5.7 Unified Framework for Benchmarking. To address the practical challenges of implementation, we propose a *Unified Benchmarking Framework*. This framework unifies isolated improvements into a coherent lifecycle, extending the rigorous construction guidelines established by recent work [19], to address the specific demands of modern Code Intelligence. The framework consists of four phases:

Phase 1: Dynamic Sourcing. To combat data leakage and ensure relevance, sourcing must move beyond static snapshots. We propose traceable sourcing [19] combined with dynamic versioning, where benchmarks are explicitly versioned against specific library updates. Furthermore, sourcing pipelines must implement cutoff-aware validation, automatically filtering repositories based on timestamps to prevent contamination.

Phase 2: Construction and Verification. Ensuring data quality at scale requires automated procedures. Benchmarks should apply deduplication and denoising to remove redundant or low-quality samples. For multi-modal tasks, additional checks are needed to verify cross-modal consistency, such as confirming that generated code correctly renders the intended user interface, thereby aligning textual correctness with visual accuracy.

Phase 3: Reproducible Evaluation Protocols. To reduce evaluation inconsistencies, benchmarks should ensure reproducible execution by distributing complete hardware and software setups, such as Docker containers, rather than isolated code snippets. In addition, evaluation should consider how a model arrives at a solution, including its use of tools and intermediate steps, instead of focusing only on the final output.

Phase 4: Sustainable Maintenance. To reduce maintenance overhead, benchmarks should move away from one-time releases toward continuous updating. Automated processes can periodically check for data contamination and collect new test cases from active repositories, helping keep benchmarks up to date with minimal manual effort.

Answer to RQ5:

Building on the identified shortcomings in RQ4, there are several promising directions for strengthening future benchmarking datasets. Incorporating multi-modality represents a valuable opportunity for developing new and more comprehensive benchmarks. Expanding benchmarks to cover more technical and specialized domains, such as DevOps, is also essential. Additionally, there is a clear need for larger-scale, real-world datasets aligned with industry demands, featuring more complex dependencies and better coverage of low-resource programming languages. As LLMs continue to advance, it is crucial to design benchmarks that can effectively evaluate their reasoning capabilities. Developing dynamic benchmarks that can be updated frequently, including after the training cut-off dates of new models, will help ensure continued relevance. Finally, greater emphasis should be placed on creating high-quality test cases and establishing more robust evaluation protocols to enable a more reliable and confident assessment of model performance.

5 Threats to Validity

External Validity. One of the main threats to the external validity of our study lies in its scope and specificity. We deliberately focused on collecting datasets explicitly designed to benchmark LLMs. To ensure this focus, we include only benchmarks developed after the emergence of LLMs (that is, early 2020) and specifically intended for their evaluation.

Internal Validity. A key concern in our data collection process is the potential for bias that arises from subjective interpretation. This bias stems primarily from unavoidable disagreements during the paper classification stage, since two researchers collaboratively categorized articles according to their titles, abstracts, and full content. To mitigate this issue and ensure quality, a third researcher ruled out disagreements that could not be resolved between the two, helping to reduce classification bias and improve consistency.

Construct Validity. The primary threat to the construct validity of our survey concerns the completeness of our dataset collection and whether we have comprehensively covered all relevant LLM benchmarking datasets. To ensure coverage, we systematically searched the major repositories of code intelligence, software engineering, and artificial intelligence, including IEEE Xplore, ACM Digital Library, ScienceDirect, and Google Scholar, and selected our candidates from an initial pool of 2,377 articles. A second threat to construct validity concerns the degree of coverage achieved for each primary study. To address this, we systematically analyzed the studies from four perspectives: overall statistics, construction processes and quality, evaluation pipelines, and shortcomings.

A third threat concerns the reliability of preprint studies. Although a structured quality-filtering process was applied (as detailed in Section 3.3), arXiv preprints had not undergone formal peer review at the time of inclusion. This may introduce risks regarding methodological maturity or result stability.

6 Conclusion

In this study, we conducted a systematic survey to investigate the characteristics of LLM benchmark datasets in code intelligence. We utilized a custom web scraper to extract the initial set of studies from four widely used online digital libraries (i.e., ACM Digital Library, IEEE Xplore, ScienceDirect, and Google Scholar). After manually filtering out studies not related to LLM benchmarking, we identified 156 benchmarks reported in 142 papers. We then developed taxonomies and systematically addressed each research question.

Our analysis reveals a notable increase in the number of benchmark datasets after 2023. We observed that 46% of the studies sourced their data from GitHub, while 26% involved the manual creation of the dataset. OpenAI's GPT models were the most widely used LLMs, featured in 85% of the studies. Regarding quality assurance, 32% of the datasets used manual inspection or expert review, while 43% lacked any formal quality control measures. Moreover, 49% of the datasets aimed to ensure relevance in the real world by collecting data from practical

sources such as GitHub and LeetCode. Execution-based evaluation emerged as the most popular methodology for assessing LLM-generated code, followed by similarity-based evaluation.

Finally, we identified and reported limitations for a representative set of datasets and proposed future directions to address these challenges in the context of LLM benchmarking for coding intelligence. This comprehensive survey aims to bridge existing gaps and provide valuable information to researchers and practitioners, providing a clearer understanding of the current landscape and future opportunities in this evolving area.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Mohamed Aghzal, Erion Plaku, Gregory J Stein, and Ziyu Yao. 2025. A survey on large language models for automated planning. *arXiv preprint arXiv:2502.12435* (2025).
- [3] Wasi Uddin Ahmad, Sean Narenthiran, Somshubra Majumdar, Aleksander Ficek, Siddhartha Jain, Jocelyn Huang, Vahid Noroozi, and Boris Ginsburg. 2025. Opencodereasoning: Advancing data distillation for competitive coding. *arXiv preprint arXiv:2504.01943* (2025).
- [4] Wasi Uddin Ahmad, Md Golam Rahman Tushar, Saikat Chakraborty, and Kai-Wei Chang. 2023. AVATAR: A Parallel Corpus for Java-Python Program Translation. In *ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). 2268–2281.
- [5] Md Basim Uddin Ahmed, Nima Shiri Harzevili, Jiho Shin, Hung Viet Pham, and Song Wang. 2025. SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection. *arXiv:2505.19828* [cs.SE] <https://arxiv.org/abs/2505.19828>
- [6] Toufique Ahmed, Christian Bird, Premkumar Devanbu, and Saikat Chakraborty. 2024. Studying LLM Performance on Closed- and Open-source Data. *arXiv:2402.15100* [cs.SE] <https://arxiv.org/abs/2402.15100>
- [7] Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. 2024. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *arXiv preprint arXiv:2412.02883* (2024).
- [8] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. Swe-bench+: Enhanced coding benchmark for llms. *arXiv preprint arXiv:2410.06992* (2024).
- [9] Miltiadis Allamanis, Sheena Panthaplackel, and Pengcheng Yin. 2024. Unsupervised evaluation of code llms with round-trip correctness. *arXiv preprint arXiv:2402.08699* (2024).
- [10] Avinash Anand, Akshit Gupta, Nishchay Yadav, and Shaurya Bajaj. 2024. A Comprehensive Survey of AI-Driven Advancements and Techniques in Automated Program Repair and Code Generation. *arXiv preprint arXiv:2411.07586* (2024).
- [11] Gábor Antal, Norbert Vándor, István Kolláth, Balázs Mosolygó, Péter Hegedűs, and Rudolf Ferenc. 2024. PyBugHive: A Comprehensive Database of Manually Validated, Reproducible Python Bugs. *IEEE Access* 12 (2024), 123739–123756.
- [12] Sima Arasteh, Georgios Nikitopoulos, Wei-Cheng Wu, Nicolaas Weideman, Aaron Portnoy, Mukund Raghothaman, and Christophe Hauser. 2025. BinPool: A Dataset of Vulnerabilities for Binary Security Analysis. *arXiv:2504.19055* [cs.CR]
- [13] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. 2022. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868* (2022).
- [14] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [15] Vaishnavi Bhargava, Rajat Ghosh, and Debojyoti Dutta. 2024. CPP-UT-Bench: Can LLMs Write Complex Unit Tests in C++? *arXiv preprint arXiv:2412.02735* (2024).
- [16] Egor Bogomolov, Aleksandra Eliseeva, Timur Galimzyanov, Evgeniy Glukhov, Anton Shapkin, Maria Tigina, Yaroslav Golubev, Alexander Kovrigin, Arie van Deursen, Maliheh Izadi, et al. 2024. Long code arena: a set of benchmarks for long-context code models. *arXiv preprint arXiv:2406.11612* (2024).
- [17] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).
- [18] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *NeurIPS 2020* 33 (2020), 1877–1901.
- [19] Jialun Cao, Yuk-Kit Chan, Zixuan Ling, Wenxuan Wang, Shuqing Li, Mingwei Liu, Ruixi Qiao, Yuting Han, Chaozheng Wang, Boxi Yu, et al. 2025. How Should We Build A Benchmark? Revisiting 274 Code-Related Benchmarks For LLMs. (2025).
- [20] Jialun Cao, Zhiyong Chen, Jiarong Wu, Shing-Chi Cheung, and Chang Xu. 2024. JavaBench: A Benchmark of Object-Oriented Code Generation for Evaluating Large Language Models. In *ASE'24 (ASE '24)*. 870–882.
- [21] Federico Cassano, John Gouwar, Francesca Lucchetti, Claire Schlesinger, Anders Freeman, Carolyn Jane Anderson, Molly Q Feldman, Michael Greenberg, Abhinav Jangda, and Arjun Guha. 2024. Knowledge transfer from high-resource to low-resource programming languages for code llms. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 677–708.

- [22] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. 2023. MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3675–3691.
- [23] Linzheng Chai, Shukai Liu, Jian Yang, Yuwei Yin, Ke Jin, Jiaheng Liu, Tao Sun, Ge Zhang, Changyu Ren, Hongcheng Guo, et al. 2024. Mceval: Massively multilingual code evaluation. *arXiv preprint arXiv:2406.07436* (2024).
- [24] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. 2024. Mle-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095* (2024).
- [25] Shubham Chandel, Colin B Clement, Guillermo Serrato, and Neel Sundaresan. 2022. Training and evaluating a jupyter notebook data science assistant. *arXiv preprint arXiv:2201.12901* (2022).
- [26] Junkai Chen, Zhiyuan Pan, Xing Hu, Zhenhao Li, Ge Li, and Xin Xia. 2024. Reasoning runtime behavior of a program with llm: How far are we? *arXiv preprint arXiv:2403.16437* (2024).
- [27] Jiachi Chen, Qingyuan Zhong, Yanlin Wang, Kaiwen Ning, Yongkun Liu, Zenan Xu, Zhe Zhao, Ting Chen, and Zibin Zheng. 2024. RMCBench: Benchmarking Large Language Models’ Resistance to Malicious Code. In *ASE’24 (ASE ’24)*. 995–1006.
- [28] Liguang Chen, Qi Guo, Hongrui Jia, Zhengran Zeng, Xin Wang, Yijiang Xu, Jian Wu, Yidong Wang, Qing Gao, Jindong Wang, et al. 2024. A survey on evaluating large language models in code generation tasks. *arXiv preprint arXiv:2408.16498* (2024).
- [29] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [30] Simin Chen, Yiming Chen, Zexin Li, Yifan Jiang, Zhongwei Wan, Yixin He, Dezhi Ran, Tianle Gu, Haizhou Li, Tao Xie, et al. 2025. Recent advances in large language model benchmarks against data contamination: From static to dynamic evaluation. *arXiv preprint arXiv:2502.17521* (2025).
- [31] Simin Chen, Pranav Pusarla, and Baishakhi Ray. 2025. DyCodeEval: Dynamic Benchmarking of Reasoning Capabilities in Code Large Language Models Under Data Contamination. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=3BZyQqbyTZ>
- [32] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *FSE’24*. 572–576.
- [33] Alireza Daghighifarsodeh, Chung-Yu Wang, Hamed Taherkhani, Melika Sepidband, Mohammad Abdollahi, Hadi Hemmati, and Hung Viet Pham. 2025. Deep-Bench: Deep Learning Benchmark Dataset for Code Generation. *arXiv preprint arXiv:2502.18726* (2025).
- [34] Jianbo Dai, Jianqiao Lu, Yunlong Feng, Dong Huang, Guangtao Zeng, Rongju Ruan, Ming Cheng, Haochen Tan, and Zhijiang Guo. 2024. Mhpp: Exploring the capabilities and limitations of language models beyond basic code generation. *arXiv preprint arXiv:2405.11430* (2024).
- [35] Zhenlong Dai, Bingrui Chen, Zhuoluo Zhao, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. *arXiv:2503.06510 [cs.SE]* <https://arxiv.org/abs/2503.06510>
- [36] Chunyuan Deng, Yilun Zhao, Yitong Li, Jiannan Cao, Xiangru Tang, and Arman Cohan. 2024. Unveiling the spectrum of data contamination in language models: A survey from detection to remediation. *arXiv preprint arXiv:2406.14644* (2024).
- [37] Ken Deng, Jiaheng Liu, He Zhu, Congnan Liu, Jingxin Li, Jiakai Wang, Peng Zhao, Chenchen Zhang, Yanan Wu, Xueqiao Yin, et al. 2024. R2c2-coder: Enhancing and benchmarking real-world repository-level code completion abilities of code large language models. *arXiv preprint arXiv:2406.01359* (2024).
- [38] Shihan Deng, Weikai Xu, Hongda Sun, Wei Liu, Tao Tan, Liujianfeng Liujianfeng, Ang Li, Jian Luan, Bin Wang, Rui Yan, and Shuo Shang. 2024. Mobile-Bench: An Evaluation Benchmark for LLM-based Mobile Agents. In *ACL’24*. 8813–8831.
- [39] Mucong Ding, Chenghao Deng, Jocelyn Choo, Zichu Wu, Aakriti Agrawal, Avi Schwarzschild, Tianyi Zhou, Tom Goldstein, John Langford, Animashree Anandkumar, et al. 2024. Easy2Hard-Bench: Standardized Difficulty Labels for Profiling LLM Performance and Generalization. *Advances in Neural Information Processing Systems* 37 (2024), 44323–44365.
- [40] Yangruibo Ding, Zijian Wang, Wasi Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, et al. 2023. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *Advances in Neural Information Processing Systems* 36 (2023), 46701–46723.
- [41] Mingzhe Du, Luu Anh Tuan, Bin Ji, Qian Liu, and See-Kiong Ng. 2024. Mercury: A Code Efficiency Benchmark for Code Large Language Models. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 16601–16622. https://proceedings.neurips.cc/paper_files/paper/2024/file/1df1df43b58845650b8dada00fca9772-Paper-Datasets_and_Benchmarks_Track.pdf
- [42] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *arXiv preprint arXiv:2308.01861* (2023).
- [43] Richard A. Dubniczky, Krisztofer Zoltán Horvát, Tamás Bisztray, Mohamed Amine Ferrag, Lucas C. Cordeiro, and Norbert Tihanyi. 2025. CASTLE: Benchmarking Dataset for Static Code Analyzers and LLMs towards CWE Detection. *arXiv:2503.09433 [cs.CR]*

- <https://arxiv.org/abs/2503.09433>
- [44] Staffs Keele et al. 2007. Guidelines for performing systematic literature reviews in software engineering. *Report, Ver. 2.3 EBSE Technical Report. EBSE.* (2007).
 - [45] Sarah Fakhoury, Saikat Chakraborty, Madanlal Musuvathi, and Shuvendu K. Lahiri. 2024. NL2Fix: Generating Functionally Correct Code Edits from Bug Descriptions. In *ICSE-Companion '24* (Lisbon, Portugal) (*ICSE-Companion '24*). 410–411.
 - [46] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering* (2024).
 - [47] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In *ICSE-FoSE'23*. 31–53.
 - [48] Jia Feng, Jiachen Liu, Cuiyun Gao, Chun Yong Chong, Chaozheng Wang, Shan Gao, and Xin Xia. 2024. ComplexCodeEval: A Benchmark for Evaluating Large Code Models on More Complex Code. In *ASE '24 (ASE '24)*. 1895–1906.
 - [49] Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, and Shubho Sengupta. 2025. Mutation-Guided LLM-based Test Generation at Meta. *arXiv preprint arXiv:2501.12862* (2025).
 - [50] Andris Freimanis and Patrick Andersson Rhodin. 2024. TableAnalyst: an LLM-agent for tabular data analysis-Implementation and evaluation on tasks of varying complexity. (2024).
 - [51] Timur Galimzyanov, Sergey Titov, Yaroslav Golubev, and Egor Bogomolov. 2024. Drawing Pandas: A Benchmark for LLMs in Generating Plotting Code. *arXiv preprint arXiv:2412.02764* (2024).
 - [52] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
 - [53] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. 2024. CRUXEval: a benchmark for code reasoning, understanding and execution. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (*ICML '24*). JMLR.org, Article 659, 54 pages.
 - [54] Ken Gu, Ruoxi Shang, Ruian Jiang, Keying Kuang, Richard-John Lin, Donghe Lyu, Yue Mao, Youran Pan, Teng Wu, Jiaqian Yu, et al. 2024. Blade: Benchmarking language model agents for data-driven science. *arXiv preprint arXiv:2408.09667* (2024).
 - [55] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
 - [56] Mohammad Kasra Habib, Daniel Graziotin, and Stefan Wagner. 2025. ReqBrain: Task-Specific Instruction Tuning of LLMs for AI-Assisted Requirements Generation. *arXiv:2505.17632 [cs.SE]* <https://arxiv.org/abs/2505.17632>
 - [57] Nam Le Hai, Dung Manh Nguyen, and Nghi D. Q. Bui. 2025. On the Impacts of Contexts on Repository-Level Code Generation. In *NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). 1496–1524.
 - [58] Hojae Han, seung-won hwang, Rajhans Samdani, and Yuxiong He. 2025. ConvCodeWorld: Benchmarking Conversational Code Generation in Reproducible Feedback Environments. In *ICLR'25*.
 - [59] Yewei Han and Chen Lyu. 2025. Multi-stage guided code generation for Large Language Models. *Engineering Applications of Artificial Intelligence* 139 (2025), 109491.
 - [60] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, J. Vanschoren and S. Yeung (Eds.), Vol. 1.
 - [61] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
 - [62] Ruida Hu, Chao Peng, Jingyi Ren, Bo Jiang, Xiangxin Meng, Qinyun Wu, Pengfei Gao, Xinchun Wang, and Cuiyun Gao. 2024. CodeRepoQA: A Large-scale Benchmark for Software Engineering Question Answering. *arXiv preprint arXiv:2412.14764* (2024).
 - [63] Wenhao Hu, Jinhao Duan, Chunchen Wei, Li Zhang, Yue Zhang, and Kaidi Xu. 2025. DynaCode: A Dynamic Complexity-Aware Code Benchmark for Evaluating Large Language Models in Code Generation. *arXiv:2503.10452 [cs.CL]* <https://arxiv.org/abs/2503.10452>
 - [64] Xing Hu, Feifei Niu, Junkai Chen, Xin Zhou, Junwei Zhang, Junda He, Xin Xia, and David Lo. 2025. Assessing and Advancing Benchmarks for Evaluating Large Language Models in Software Engineering Tasks. *arXiv:2505.08903 [cs.SE]* <https://arxiv.org/abs/2505.08903>
 - [65] Dong Huang, Jie M Zhang, Yuhao Qing, and Heming Cui. 2024. EfficBench: Benchmarking the Efficiency of Automatically Generated Code. *CoRR abs/2402.02037* (2024).
 - [66] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
 - [67] Yuan Huang, Yinan Chen, Xiangping Chen, Junqi Chen, Rui Peng, Zhicao Tang, Jinbo Huang, Furen Xu, and Zibin Zheng. 2024. Generative Software Engineering. *CoRR abs/2403.02583* (2024). <https://doi.org/10.48550/ARXIV.2403.02583> *arXiv:2403.02583*

- [68] Yiming Huang, Jianwen Luo, Yan Yu, Yitong Zhang, Fangyu Lei, Yifan Wei, Shizhu He, Lifu Huang, Xiao Liu, Jun Zhao, and Kang Liu. 2024. DA-Code: Agent Data Science Code Generation Benchmark for Large Language Models. In *EMNLP'24*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). 13487–13521.
- [69] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- [70] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2020. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *arXiv:1909.09436 [cs.LG]* <https://arxiv.org/abs/1909.09436>
- [71] Kush Jain, Gabriel Synnaeve, and Baptiste Rozière. 2024. TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark. *arXiv preprint arXiv:2410.00752* (2024).
- [72] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974* (2024).
- [73] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2022. Jigsaw: large language models meet program synthesis. In *ICSE '22* (Pittsburgh, Pennsylvania) (*ICSE '22*). 1219–1231.
- [74] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv:2310.06825 [cs.CL]* <https://arxiv.org/abs/2310.06825>
- [75] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *ICSE'23*. 1430–1442.
- [76] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the evaluation of neural code translation: Taxonomy and benchmark. In *ASE'23*. IEEE, 1529–1541.
- [77] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).
- [78] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From llms to llm-based agents for software engineering: A survey of current, challenges and future. *arXiv preprint arXiv:2408.02479* (2024).
- [79] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2024. DS-Bench: How Far Are Data Science Agents to Becoming Data Science Experts? *arXiv preprint arXiv:2409.07703* (2024).
- [80] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (San Jose, CA, USA) (*ISTTA 2014*). Association for Computing Machinery, New York, NY, USA, 437–440. <https://doi.org/10.1145/2610384.2628055>
- [81] Sairam Vakkalanka Kai Petersen and Ludwik Kuzniarz. 2015. Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology* (2015).
- [82] Mohammad Abdullah Matin Khan, M Saiful Bari, Xuan Long Do, Weishi Wang, Md Rizwan Parvez, and Shafiq Joty. 2024. XCodeEval: An Execution-based Large Scale Multilingual Multitask Benchmark for Code Understanding, Generation, Translation and Retrieval. In *ACL'24*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). 6766–6805.
- [83] Roham Koohestani, Philippe de Bekker, and Maliheh Izadi. 2025. Benchmarking AI Models in Software Engineering: A Review, Search Tool, and Enhancement Protocol. *arXiv preprint arXiv:2503.05860* (2025).
- [84] Sachit Kumar, Wasi Uddin Ahmad, Zijian Wang, Nihal Jain, Haifeng Qian, Baishakhi Ray, Murali Krishna Ramanathan, Xiaofei Ma, and Anoop Deoras. 2025. LibEvolutionEval: A Benchmark and Study for Version-Specific Code Generation. In *NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.).
- [85] Ummay Kulsum, Haotian Zhu, Bowen Xu, and Marcelo d'Amorim. 2024. A case study of llm for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback. In *AIware'24*. 103–111.
- [86] Beck LaBash, August Rosedale, Alex Reents, Lucas Negritto, and Colin Wiel. 2024. RES-Q: Evaluating Code-Editing Large Language Model Systems at the Repository Scale. *arXiv preprint arXiv:2406.16801* (2024).
- [87] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: a natural and reliable benchmark for data science code generation. In *ICML'23* (Honolulu, Hawaii, USA) (*ICML'23*). JMLR.org, Article 756, 27 pages.
- [88] Cheryl Lee, Chunqiu Steven Xia, Longji Yang, Jen-tse Huang, Zhouyixin Zhu, Lingming Zhang, and Michael R Lyu. 2024. A unified debugging approach via llm-based multi-agent synergy. *arXiv preprint arXiv:2404.17153* (2024).
- [89] Jae Yong Lee, Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2024. The GitHub Recent Bugs Dataset for Evaluating LLM-Based Debugging Applications. In *ICST'24*. 442–444.
- [90] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, Zhiyin Yu, He Du, Ping Yang, Dahua Lin, Chao Peng, and Kai Chen. 2025. Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow,

- Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). 7511–7531.
- [91] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C.C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs. In *NIPS '23* (New Orleans, LA, USA) (*NIPS '23*). Curran Associates Inc., Red Hook, NY, USA, Article 1835, 28 pages.
 - [92] Jinyang Li, Nan Huo, Yan Gao, Jiayi Shi, Yingxiu Zhao, Ge Qu, Yurong Wu, Chenhao Ma, Jian-Guang Lou, and Reynold Cheng. 2024. Tapilot-crossing: benchmarking and evolving LLMs towards interactive data analysis agents. *arXiv preprint arXiv:2403.05307* (2024).
 - [93] Jia Li, Ge Li, Xuanming Zhang, Yihong Dong, and Zhi Jin. 2024. Evocodebench: An evolving code generation benchmark aligned with real-world code repositories. *arXiv preprint arXiv:2404.00599* (2024).
 - [94] Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Huanyu Liu, Hao Zhu, Lecheng Wang, Kaibo Liu, Zheng Fang, Lanshen Wang, Jiazheng Ding, Xuanming Zhang, Yuqi Zhu, Yihong Dong, Zhi Jin, Binhua Li, Fei Huang, Yongbin Li, Bin Gu, and Mengfei Yang. 2024. DevEval: A Manually-Annotated Code Generation Benchmark Aligned with Real-World Code Repositories. In *ACL 2024*. 3603–3614.
 - [95] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *EMNLP 2023*. 3102–3116.
 - [96] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
 - [97] Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. 2023. Taco: Topics in algorithmic code generation dataset. *arXiv preprint arXiv:2312.14852* (2023).
 - [98] Shangyu Li, Juyong Jiang, Tiancheng Zhao, and Jiasi Shen. 2025. OSVBench: Benchmarking LLMs on Specification Generation Tasks for Operating System Verification. *arXiv:2504.20964* [cs.CL] <https://arxiv.org/abs/2504.20964>
 - [99] Wei Li, Xin Zhang, Zhongxin Guo, Shaoguang Mao, Wen Luo, Guangyue Peng, Yangyu Huang, Houfeng Wang, and Scarlett Li. 2025. FEA-Bench: A Benchmark for Evaluating Repository-Level Code Generation for Feature Implementation. *arXiv:2503.06680* [cs.SE] <https://arxiv.org/abs/2503.06680>
 - [100] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
 - [101] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. *arXiv:2405.17238* [cs.CR] <https://arxiv.org/abs/2405.17238>
 - [102] Shanchao Liang, Yiran Hu, Nan Jiang, and Lin Tan. 2025. Can Language Models Replace Programmers for Coding? REPOCOD Says 'Not Yet'. *arXiv:2410.21647* [cs.SE] <https://arxiv.org/abs/2410.21647>
 - [103] Shanchao Liang, Nan Jiang, Shangshu Qian, and Lin Tan. 2024. WAFFLE: Multi-Modal Model for Automated Front-End Development. *arXiv preprint arXiv:2410.18362* (2024).
 - [104] Dianshu Liao, Shidong Pan, Xiaoyu Sun, Xiaoxue Ren, Qing Huang, Zhenchang Xing, Huan Jin, and Qinying Li. 2024. A³A3-CodGen: A Repository-Level Code Generation Framework for Code Reuse With Local-Aware, Global-Aware, and Third-Party-Library-Aware. *IEEE Trans. Softw. Eng.* 50, 12 (Dec. 2024), 3369–3384.
 - [105] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. QuixBugs: a multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity* (Vancouver, BC, Canada) (*SPLASH Companion 2017*). Association for Computing Machinery, New York, NY, USA, 55–56. <https://doi.org/10.1145/3135932.3135941>
 - [106] Changshu Liu and Reyhaneh Jabbarvand. 2025. A Tool for In-depth Analysis of Code Execution Reasoning of Large Language Models. *arXiv preprint arXiv:2501.18482* (2025).
 - [107] Chunhua Liu, Hong Yi Lin, and Patanamon Thongtanunam. 2025. Too Noisy To Learn: Enhancing Data Quality for Code Review Comment Generation. *arXiv:2502.02757* [cs.SE] <https://arxiv.org/abs/2502.02757>
 - [108] Changshu Liu, Shizhuo Dylan Zhang, Ali Reza Ibrahimzada, and Reyhaneh Jabbarvand. 2024. Codemind: A framework to challenge large language models for code reasoning. *arXiv preprint arXiv:2402.09664* (2024).
 - [109] Ethan TS. Liu, Austin Wang, Spencer Mateega, Carlos Georgescu, and Danny Tang. 2025. VADER: A Human-Evaluated Benchmark for Vulnerability Assessment, Detection, Explanation, and Remediation. *arXiv:2505.19395* [cs.CR] <https://arxiv.org/abs/2505.19395>
 - [110] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572.
 - [111] Jiawei Liu, Songrun Xie, Junhao Wang, Yuxiang Wei, Yifeng Ding, and Lingming Zhang. 2024. Evaluating language models for efficient code generation. *arXiv preprint arXiv:2408.06450* (2024).
 - [112] Jun Liu, Jiwei Yan, Yuanyuan Xie, Jun Yan, and Jian Zhang. 2024. Fix the Tests: Augmenting LLMs to Repair Test Cases with Static Collector and Neural Reranker. In *ISSRE'23*. 367–378. <https://doi.org/10.1109/ISSRE62328.2024.00043>
 - [113] Kaiyuan Liu, Youcheng Pan, Yang Xiang, Daojing He, Jing Li, Yexing Du, and Tianrun Gao. 2025. ProjectEval: A Benchmark for Programming Agents Automated Evaluation on Project-Level Code Generation. *arXiv:2503.07010* [cs.SE] <https://arxiv.org/abs/2503.07010>

- [114] Shukai Liu, Linzheng Chai, Jian Yang, Jiajun Shi, He Zhu, Liran Wang, Ke Jin, Wei Zhang, Hualei Zhu, Shuyue Guo, et al. 2024. Mdeval: Massively multilingual code debugging. *arXiv preprint arXiv:2411.02310* (2024).
- [115] Siyao Liu, He Zhu, Jerry Liu, Shulin Xin, Aoyan Li, Rui Long, Li Chen, Jack Yang, Jinxiang Xia, ZY Peng, et al. 2024. Fullstack bench: Evaluating llms as full stack coder. *arXiv preprint arXiv:2412.00535* (2024).
- [116] Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. <https://arxiv.org/abs/2306.03091>
- [117] Chloe Loughridge, Qinyi Sun, Seth Ahrenbach, Federico Cassano, Chuyue Sun, Ying Sheng, Anish Mudide, Md Rakib Hossain Misu, Nada Amin, and Max Tegmark. 2024. DafnyBench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467* (2024).
- [118] Chaomeng Lu, Tianyu Li, Toon Dehaene, and Bert Lagaisse. 2025. ICVul: A Well-labeled C/C++ Vulnerability Dataset with Comprehensive Metadata and VCCs. In *MSR'25*. IEEE, 154–158.
- [119] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, et al. 2024. Toolandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. *arXiv preprint arXiv:2408.04682* (2024).
- [120] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021).
- [121] Lezhi Ma, Shangqing Liu, Lei Bu, Shangru Li, Yida Wang, and Yang Liu. 2024. SpecEval: Evaluating Code Comprehension in Large Language Models via Program Specifications. *arXiv preprint arXiv:2409.12866* (2024).
- [122] Alexandre Matton, Tom Sherborne, Dennis Aumiller, Elena Tommasone, Milad Alizadeh, Jingyi He, Raymond Ma, Maxime Voisin, Ellen Gilsenan-McMahon, and Matthias Gallé. 2024. On Leakage of Code Generation Evaluation Datasets. In *EMNLP 2024*. 13215–13223.
- [123] Behzad Mehrbakhsh, Dario Garigliotti, Fernando Martínez-Plumed, and Jose Hernandez-Orallo. 2024. Confounders in instance variation for the analysis of data contamination. In *Proceedings of the 1st Workshop on Data Contamination (CONDA)*. 13–21.
- [124] Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. 2025. SWE-Lancer: Can Frontier LLMs Earn \$1 Million from Real-World Freelance Software Engineering?. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=xZXhFg43EI>
- [125] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binqun Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. 2023. Clarifygpt: Empowering llm-based code generation with intention clarification. *arXiv preprint arXiv:2310.10996* (2023).
- [126] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 81857–81887.
- [127] Noor Nashid, Islem Bouzenia, Michael Pradel, and Ali Mesbah. 2025. Issue2Test: Generating Reproducing Test Cases from Issue Reports. *arXiv preprint arXiv:2503.16320* (2025).
- [128] Dung Manh Nguyen, Thang Chau Phan, Nam Le Hai, Tien-Thong Doan, Nam V Nguyen, Quang Pham, and Nghi DQ Bui. 2024. CodeMMLU: A Multi-Task Benchmark for Assessing Code Understanding & Reasoning Capabilities of CodeLLMs. In *ICLR'24*.
- [129] Minh Huynh Nguyen, Thang Phan Chau, Phong X Nguyen, and Nghi DQ Bui. 2024. Agilecoder: Dynamic collaborative agents for software development based on agile methodology. *arXiv preprint arXiv:2406.11912* (2024).
- [130] Daniel Nichols, Joshua H Davis, Zhaojun Xie, Arjun Rajaram, and Abhinav Bhatele. 2024. Can large language models write parallel code?. In *HPDC'24*. 281–294.
- [131] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).
- [132] Changan Niu, Chuanyi Li, Vincent Ng, and Bin Luo. 2023. Crosscodebench: Benchmarking cross-task generalization of source code models. In *ICSE'23*. 537–549.
- [133] Md Nahidul Islam Opu, Shaowei Wang, and Shaiful Chowdhury. 2025. LLM-Based Detection of Tangled Code Changes for Higher-Quality Method-Level Bug Datasets. arXiv:2505.08263 [cs.SE] <https://arxiv.org/abs/2505.08263>
- [134] Guangsheng Ou, Mingwei Liu, Yuxuan Chen, Xin Peng, and Zibin Zheng. 2024. Repository-level code translation benchmark targeting rust. *arXiv preprint arXiv:2411.13990* (2024).
- [135] Wendkūni C. Ouédraogo, Kader Kaboré, Haoye Tian, Yewei Song, Anil Koyuncu, Jacques Klein, David Lo, and Tegawendé F. Bissyandé. 2024. Large-scale, Independent and Comprehensive study of the power of LLMs for test case generation. arXiv:2407.00225 [cs.SE] <https://arxiv.org/abs/2407.00225>
- [136] Ipek Ozkaya. 2023. Application of Large Language Models to Software Engineering Tasks: Opportunities, Risks, and Implications. *IEEE Software* 40, 3 (2023), 4–8. <https://doi.org/10.1109/MS.2023.3248401>
- [137] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. [n. d.]. Lost in translation: A study of bugs introduced by large language models while translating code. In *ICSE'24*. 1–13.

- [138] Zhenyu Pan, Rongyu Cao, Yongchang Cao, Yingwei Ma, Binhua Li, Fei Huang, Han Liu, and Yongbin Li. 2024. Codev-Bench: How Do LLMs Understand Developer-Centric Code Completion? *arXiv preprint arXiv:2410.01353* (2024).
- [139] Debalina Ghosh Paul, Hong Zhu, and Ian Bayley. 2024. ScenEval: A Benchmark for Scenario-Based Evaluation of Code Generation. In *AITest'24*. 55–63.
- [140] Qiwei Peng, Yekun Chai, and Xuhong Li. 2024. HumanEval-XL: A Multilingual Code Generation Benchmark for Cross-lingual Natural Language Generalization. In *LREC-COLING 2024*. 8383–8394.
- [141] Yun Peng, Jun Wan, Yichen Li, and Xiaoxue Ren. 2025. COFFE: A Code Efficiency Benchmark for Code Generation. *arXiv:2502.02827* [cs.SE] <https://arxiv.org/abs/2502.02827>
- [142] Zhiyuan Peng, Xin Yin, Rui Qian, Peiqin Lin, Yongkang Liu, Chenhao Ying, and Yuan Luo. 2025. SolEval: Benchmarking Large Language Models for Repository-level Solidity Code Generation. *arXiv:2502.18793* [cs.SE] <https://arxiv.org/abs/2502.18793>
- [143] Shanghaoran Quan, Jiaxi Yang, Bowen Yu, Bo Zheng, Dayiheng Liu, An Yang, Xuancheng Ren, Bofei Gao, Yibo Miao, Yunlong Feng, et al. 2025. CodeElo: Benchmarking Competition-level Code Generation of LLMs with Human-comparable Elo Ratings. *arXiv preprint arXiv:2501.01257* (2025).
- [144] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [145] Shanto Rahman, Sachit Kuhar, Berk Cirisci, Pranav Garg, Shiqi Wang, Xiaofei Ma, Anoop Deoras, and Baishakhi Ray. 2025. UTFix: Change Aware Unit Test Repairing using LLM. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 85 (April 2025), 26 pages. <https://doi.org/10.1145/3720419>
- [146] Nishat Raihan, Antonios Anastasopoulos, and Marcos Zampieri. 2025. mHumanEval - A Multilingual Benchmark to Evaluate Large Language Models for Code Generation. In *NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.).
- [147] Tal Ridnik, Dedy Kredo, and Itamar Friedman. 2024. Code generation with alphacodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500* (2024).
- [148] Manley Roberts, Himanshu Thakur, Christine Herlihy, Colin White, and Samuel Dooley. 2023. To the cutoff... and beyond? a longitudinal perspective on llm data contamination. In *The Twelfth International Conference on Learning Representations*.
- [149] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [150] Bonan Ruan, Jiahao Liu, Weibo Zhao, and Zhenkai Liang. 2024. VulZoo: A Comprehensive Vulnerability Intelligence Dataset. *arXiv:2406.16347* [cs.CR] <https://arxiv.org/abs/2406.16347>
- [151] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 951–971.
- [152] Ahmadreza Saboor Yaraghi, Darren Holden, Nafiseh Kahani, and Lionel Briand. 2025. Automated Test Case Repair Using Language Models. *IEEE Trans. Softw. Eng.* 51, 4 (Feb. 2025), 1104–1133. <https://doi.org/10.1109/TSE.2025.3541166>
- [153] Soumit Kanti Saha, Fazle Rabbi, Song Wang, and Jinjia Yang. 2024. Specification-Driven Code Translation Powered by Large Language Models: How Far Are We? *arXiv preprint arXiv:2412.04590* (2024).
- [154] Nidhish Shah, Zulkuf Genc, and Dogu Araci. 2024. StackEval: Benchmarking LLMs in Coding Assistance. *Advances in Neural Information Processing Systems* 37 (2024), 36976–36994.
- [155] Jiho Shin, Clark Tang, Tahmineh Mohati, Maleknaz Nayebi, Song Wang, and Hadi Hemmati. 2023. Prompt engineering or fine tuning: An empirical assessment of large language models in automated software engineering tasks. *arXiv preprint arXiv:2310.10508* (2023).
- [156] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652.
- [157] Mohammed Latif Siddiq, Simantika Dristi, Joy Saha, and Joanna CS Santos. 2024. The Fault in our Stars: Quality Assessment of Code Generation Benchmarks. In *2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 201–212.
- [158] André Silva and Martin Monperrus. 2024. *RepairBench: Leaderboard of Frontier Models for Program Repair*. Technical Report 2409.18952. *arXiv*. <https://arxiv.org/abs/2409.18952>
- [159] Manav Singhal, Tushar Aggarwal, Abhijeet Awasthi, Nagarajan Natarajan, and Aditya Kanade. 2024. Nofuneval: Funny how code lms falter on requirements beyond functional correctness. *arXiv preprint arXiv:2401.15963* (2024).
- [160] Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang, Cheng Li, Ke Wang, Rong Yao, et al. 2023. Restgpt: Connecting large language models with real-world restful apis. *arXiv preprint arXiv:2306.06624* (2023).
- [161] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Wei Ma, Lyuye Zhang, Yang Liu, and Yingjiu Li. 2025. LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning. *arXiv:2401.16185* [cs.CR] <https://arxiv.org/abs/2401.16185>
- [162] Boyin Tan, Junjielong Xu, Zhouxiang Zhu, and Pinjia He. 2025. AL-Bench: A Benchmark for Automatic Logging. *arXiv:2502.03160* [cs.SE] <https://arxiv.org/abs/2502.03160>
- [163] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Hui Haotian, Liu Weichuan, Zhiyuan Liu, and Maosong Sun. 2024. DebugBench: Evaluating Debugging Capability of Large Language Models. In *ACL 2024*. 4173–4198.

- [164] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [165] Mansi Uniyal, Mukul Singh, Gust Verbruggen, Sumit Gulwani, and Vu Le. 2024. One-to-many testing for code generation from (just) natural language. In *EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 15397–15402. <https://doi.org/10.18653/v1/2024.findings-emnlp.902>
- [166] Gianmario Voria, Francesco Casillo, Carmine Gravino, Gemma Catolino, and Fabio Palomba. 2025. RECOVER: Toward Requirements Generation From Stakeholders' Conversations. *IEEE Transactions on Software Engineering* 51, 06 (June 2025), 1912–1933. <https://doi.org/10.1109/TSE.2025.3572056>
- [167] Hanbin Wang, Xiaoxuan Zhou, Zhipeng Xu, Keyuan Cheng, Yuxin Zuo, Kai Tian, Jingwei Song, Junting Lu, Wenhui Hu, and Xueyang Liu. 2025. Code-Vision: Evaluating Multimodal LLMs Logic Understanding and Code Generation Capabilities. *arXiv:2502.11829* [cs.CL] <https://arxiv.org/abs/2502.11829>
- [168] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* (2024).
- [169] Jize Wang, Ma Zerun, Yining Li, Songyang Zhang, Cailian Chen, Kai Chen, and Xinyi Le. 2024. GTA: a benchmark for general tool agents. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- [170] Kaixin Wang, Tianlin Li, Xiaoyu Zhang, Chong Wang, Weisong Sun, Yang Liu, and Bin Shi. 2025. Software Development Life Cycle Perspective: A Survey of Benchmarks for Code Large Language Models and Agents. *arXiv:2505.05283* [cs.SE] <https://arxiv.org/abs/2505.05283>
- [171] Peiding Wang, Li Zhang, Fang Liu, Lin Shi, Minxiao Li, Bo Shen, and An Fu. 2025. CodeIF-Bench: Evaluating Instruction-Following Capabilities of Large Language Models in Interactive Code Generation. *arXiv:2503.22688* [cs.SE] <https://arxiv.org/abs/2503.22688>
- [172] Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2025. TESTEVAL: Benchmarking Large Language Models for Test Case Generation. In *NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.).
- [173] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).
- [174] Yanli Wang, Yanlin Wang, Suiquan Wang, Daya Guo, Jiachi Chen, John Grundy, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, et al. 2024. RepoTransBench: A Real-World Benchmark for Repository-Level Code Translation. *arXiv preprint arXiv:2412.17744* (2024).
- [175] Yibo Wang, Congying Xia, Wenting Zhao, Jiangshu Du, Chunyu Miao, Zhongfen Deng, Philip S. Yu, and Chen Xing. 2025. ProjectTest: A Project-level LLM Unit Test Generation Benchmark and Impact of Error Fixing Mechanisms. *arXiv:2502.06556* [cs.SE] <https://arxiv.org/abs/2502.06556>
- [176] Zhengren Wang, Rui Ling, Chufan Wang, Yongan Yu, Sizhe Wang, Zhiyu Li, Feiyu Xiong, and Wentao Zhang. 2025. MaintainCoder: Maintainable Code Generation Under Dynamic Requirements. *arXiv:2503.24260* [cs.SE] <https://arxiv.org/abs/2503.24260>
- [177] Zihan Wang, Siyao Liu, Yang Sun, Hongyan Li, and Kai Shen. 2025. CodeContests+: High-Quality Test Case Generation for Competitive Programming. *arXiv preprint arXiv:2506.05817* (2025).
- [178] Zhiruo Wang, Shuyan Zhou, Daniel Fried, and Graham Neubig. 2023. Execution-Based Evaluation for Open-Domain Code Generation. In *EMNLP 2023*. 1271–1290.
- [179] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddhartha Naidu, et al. 2024. Livebench: A challenging, contamination-free llm benchmark. *arXiv preprint arXiv:2406.19314* (2024).
- [180] Chengyue Wu, Zhixuan Liang, Yixiao Ge, Qiushan Guo, Zeyu Lu, Jiahao Wang, Ying Shan, and Ping Luo. 2025. Plot2Code: A Comprehensive Benchmark for Evaluating Multi-modal Large Language Models in Code Generation from Scientific Plots. In *NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). 3006–3028.
- [181] Jie Jw Wu and Fatemeh H. Fard. 2025. HumanEvalComm: Benchmarking the Communication Competence of Code Generation for LLMs and LLM Agent. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2025).
- [182] Xiaobao Wu, Liangming Pan, Yuxi Xie, Ruiwen Zhou, Shuai Zhao, Yubo Ma, Mingzhe Du, Rui Mao, Anh Tuan Luu, and William Yang Wang. 2024. Antileak-bench: Preventing data contamination by automatically constructing benchmarks with updated real-world knowledge. *arXiv preprint arXiv:2412.13670* (2024).
- [183] Yonghao Wu, Zheng Li, Jie M. Zhang, and Yong Liu. 2024. ConDefects: A Complementary Dataset to Address the Data Leakage Concern for LLM-Based Fault Localization and Program Repair. In *FSE'24 (FSE 2024)*. 642–646.
- [184] Antonia Wüst, Wolfgang Stammer, Quentin Delfosse, Devendra Singh Dhami, and Kristian Kersting. 2024. Pix2code: Learning to compose neural visual concepts as programs. *arXiv preprint arXiv:2402.08280* (2024).
- [185] Chunqiu Steven Xia, Yinlin Deng, and Lingming Zhang. 2024. Top leaderboard ranking= top coding proficiency, always? evoeval: Evolving coding benchmarks via llm. *arXiv preprint arXiv:2403.19114* (2024).
- [186] Yinghui Xia, Yuyan Chen, Tianyu Shi, Jun Wang, and Jinsong Yang. 2024. Aicodereval: Improving ai domain code generation of large language models. *arXiv preprint arXiv:2406.04712* (2024).

- [187] Kai Xu, YiWei Mao, XinYi Guan, and ZiLong Feng. 2025. Web-Bench: A LLM Code Benchmark Based on Web Standards and Frameworks. *arXiv:2505.07473* [cs.AI] <https://arxiv.org/abs/2505.07473>
- [188] Ankit Yadav, Himanshu Beniwal, and Mayank Singh. 2024. PythonSaga: Redefining the Benchmark to Evaluate Code Generating LLMs. In *EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). 17113–17126.
- [189] Kaiwen Yan, Hongcheng Guo, Xuanqing Shi, Jingyi Xu, Yaonan Gu, and Zhoujun Li. 2025. CodeIF: Benchmarking the Instruction-Following Capabilities of Large Language Models for Code Generation. *arXiv:2502.19166* [cs.SE]
- [190] Weixiang Yan, Haitian Liu, Yunkun Wang, Yunzhe Li, Qian Chen, Wen Wang, Tingyu Lin, Weishan Zhao, Li Zhu, Hari Sundaram, and Shuiguang Deng. 2024. CodeScope: An Execution-based Multilingual Multitask Multidimensional Benchmark for Evaluating LLMs on Code Understanding and Generation. In *ACL'24*. 5511–5558.
- [191] Weixiang Yan, Yuchen Tian, Yunzhe Li, Qian Chen, and Wen Wang. 2023. Codetransocean: A comprehensive multilingual benchmark for code translation. *arXiv preprint arXiv:2310.04951* (2023).
- [192] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F. Bissyandé, and Shunfu Jin. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *ISSTA'24*. 882–894.
- [193] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. 2024. SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains? *arXiv preprint arXiv:2410.03859* (2024).
- [194] Jian Yang, Jiajun Zhang, Jiaxi Yang, Ke Jin, Lei Zhang, Qiyao Peng, Ken Deng, Yibo Miao, Tianyu Liu, Zeyu Cui, et al. 2024. Execrepobench: Multi-level executable code completion evaluation. *arXiv preprint arXiv:2412.11990* (2024).
- [195] Lei Yang, Renren Jin, Ling Shi, Jianxiang Peng, Yue Chen, and Deyi Xiong. 2025. Probench: Benchmarking large language models in competitive programming. *arXiv preprint arXiv:2502.20868* (2025).
- [196] Weiqing Yang, Hanbin Wang, Zhenghao Liu, Xinze Li, Yukun Yan, Shuo Wang, Yu Gu, Minghe Yu, Zhiyuan Liu, and Ge Yu. 2025. COAST: Enhancing the Code Debugging Ability of LLMs through Communicative Agent Based Data Synthesis. In *NAACL 2025*. 2570–2585.
- [197] Zhiyu Yang, Shuo Wang, Yukun Yan, and Yang Deng. 2025. Why Stop at One Error? Benchmarking LLMs as Data Science Code Debuggers for Multi-Hop and Multi-Bug Errors. *arXiv preprint arXiv:2503.22388* (2025).
- [198] Alperen Yildiz, Sin G. Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil M. Divakaran. 2025. Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories. *arXiv:2503.03586* [cs.CR] <https://arxiv.org/abs/2503.03586>
- [199] Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to Mine Aligned Code and Natural Language Pairs from Stack Overflow. *arXiv:1805.08949* [cs.CL] <https://arxiv.org/abs/1805.08949>
- [200] Pengcheng Yin, Wen-Ding Li, Kefan Xiao, Abhishek Rao, Yeming Wen, Kensen Shi, Joshua Howland, Paige Bailey, Michele Catasta, Henryk Michalewski, Oleksandr Polozov, and Charles Sutton. 2023. Natural Language to Code Generation in Interactive Data Science Notebooks. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 126–173. <https://doi.org/10.18653/v1/2023.acl-long.9>
- [201] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. [n. d.]. Thinkrepair: Self-directed automated program repair. In *ISSTA'24*. 1274–1286.
- [202] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *ICSE'24*. 1–12.
- [203] Daoguang Zan, Bei Chen, Zeqi Lin, Bei Guan, Wang Yongji, and Jian-Guang Lou. 2022. When Language Model Meets Private Library. In *EMNLP 2022*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). 277–288.
- [204] Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2022. CERT: Continual Pre-training on Sketches for Library-oriented Code Generation. In *IJCAI-22*. 2369–2375.
- [205] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. *arXiv:2504.02605* [cs.SE] <https://arxiv.org/abs/2504.02605>
- [206] Daoguang Zan, Zhirong Huang, Ailun Yu, Shaoxin Lin, Yifan Shi, Wei Liu, Dong Chen, Zongshuai Qi, Hao Yu, Lei Yu, et al. 2024. Swe-bench-java: A github issue resolving benchmark for java. *arXiv preprint arXiv:2408.14354* (2024).
- [207] Zhengran Zeng, Yidong Wang, Rui Xie, Wei Ye, and Shikun Zhang. 2024. CoderUJB: An Executable and Unified Java Benchmark for Practical Programming Scenarios. In *ISSTA 2024 (ISSTA 2024)*. 124–136.
- [208] Cong Zhang, Xin Deik Goh, Dexun Li, Hao Zhang, and Yong Liu. 2025. Planning with Multi-Constraints via Collaborative Language Agents. In *Proceedings of the 31st International Conference on Computational Linguistics*. 10054–10082.
- [209] Fengji Zhang, Linquan Wu, Huiyu Bai, Guancheng Lin, Xiao Li, Xiao Yu, Yue Wang, Bei Chen, and Jacky Keung. 2024. HumanEval-V: Evaluating Visual Understanding and Reasoning Abilities of Large Multimodal Models Through Coding Tasks. *arXiv preprint arXiv:2410.12381* (2024).

- [210] Linhao Zhang, Daoguang Zan, Quanshun Yang, Zhirong Huang, Dong Chen, Bo Shen, Tianyu Liu, Yongshun Gong, Pengjie Huang, Xudong Lu, et al. 2024. CodeV: Issue Resolving with Visual Data. *arXiv preprint arXiv:2412.17315* (2024).
- [211] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2023. A survey on large language models for software engineering. *arXiv preprint arXiv:2312.15223* (2023).
- [212] Quanjun Zhang, Ye Shang, Chunrong Fang, Siqi Gu, Jianyi Zhou, and Zhenyu Chen. 2024. TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models. *arXiv preprint arXiv:2409.17561* (2024).
- [213] Shudan Zhang, Hanlin Zhao, Xiao Liu, Qinkai Zheng, Zehan Qi, Xiaotao Gu, Xiaohan Zhang, Yuxiao Dong, and Jie Tang. 2024. NaturalCodeBench: Examining Coding Performance Mismatch on HumanEval and Natural User Prompts.(2024). URL <https://doi.org/10.48550/arXiv.2405.2405> (2024).
- [214] Ziyin Zhang, Chaoyu Chen, Bingchang Liu, Cong Liao, Zi Gong, Hang Yu, Jianguo Li, and Rui Wang. 2023. Unifying the perspectives of nlp and software engineering: A survey on language models for code. *arXiv preprint arXiv:2311.07989* (2023).
- [215] Yuwei Zhao, Ziyang Luo, Yuchen Tian, Hongzhan Lin, Weixiang Yan, Annan Li, and Jing Ma. 2024. CodeJudge-Eval: Can Large Language Models be Good Judges in Code Understanding? *arXiv preprint arXiv:2408.10718* (2024).
- [216] Jiasheng Zheng, Boxi Cao, Zhengzhao Ma, Ruotong Pan, Hongyu Lin, Yaojie Lu, Xianpei Han, and Le Sun. 2024. Beyond Correctness: Benchmarking Multi-dimensional Code Generation for Large Language Models. *arXiv preprint arXiv:2407.11470* (2024).
- [217] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. In *SIGKDD'23 (KDD '23)*. 5673–5684.
- [218] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. 2023. A survey of large language models for code: Evolution, benchmarking, and future trends. *arXiv preprint arXiv:2311.10372* (2023).
- [219] Zibin Zheng, Kaiwen Ning, Qingyuan Zhong, Jiachi Chen, Wenqing Chen, Lianghong Guo, Weicheng Wang, and Yanlin Wang. 2024. Towards an understanding of large language models in software engineering tasks. *Empirical Software Engineering* 30, 2 (2024), 50.
- [220] Li Zhong and Zilong Wang. 2024. Can LLM Replace Stack Overflow? A Study on Robustness and Reliability of Large Language Model Code Generation. 38 (Mar. 2024), 21841–21849. <https://doi.org/10.1609/aaai.v38i19.30185>
- [221] Xin Zhou, Martin Weyssow, Ratnadira Widyasari, Ting Zhang, Junda He, Yunbo Lyu, Jianming Chang, Beiqi Zhang, Dan Huang, and David Lo. 2025. LessLeak-Bench: A First Investigation of Data Leakage in LLMs Across 83 Software Engineering Benchmarks. *arXiv preprint arXiv:2502.06215* (2025).
- [222] Zhenhao Zhou, Zhuochen Huang, Yike He, Chong Wang, Jiajun Wang, Yijian Wu, Xin Peng, and Yiling Lou. 2025. Benchmarking and Enhancing LLM Agents in Localizing Linux Kernel Bugs. *arXiv:2505.19489 [cs.AI]* <https://arxiv.org/abs/2505.19489>
- [223] Hao-Nan Zhu, Robert M Furth, Michael Pradel, and Cindy Rubio-González. 2025. From Bugs to Benchmarks: A Comprehensive Survey of Software Defect Datasets. *arXiv preprint arXiv:2504.17977* (2025).
- [224] Qiming Zhu, Jialun Cao, Yaojie Lu, Hongyu Lin, Xianpei Han, Le Sun, and Shing-Chi Cheung. 2025. Domaineval: An auto-constructed benchmark for multi-domain code generation. In *AAAI'25*, Vol. 39. 26148–26156.
- [225] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. 2024. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877* (2024).